

SHOGUN

A sixteen-voice analog-style drum machine with a full patch bay, from Martial Systems.

SHOGUN has fourteen drum voices and two synth voices, a step sequencer for every voice, and a 151-jack patch bay that cables to the other instruments in the Jidai Collection. It runs as a VST3 instrument in your DAW and as a playable web page built from the same engine.

Features

Voices

- **Fourteen drums:** two kicks (BD1, BD2), snare (SD), rim (RS), clap (CP), claves (CL), maracas (MA), cowbell (CB), closed and open hats (CH, OH) with choke, cymbal (CY), and three toms that switch to congas (LTC, MTC, HTC).
- **Two synth voices:** LEAD with a state-variable filter and BASS with a 4-pole ladder filter, both with glide and ties.
- **Triple wave shaper** on the kicks and toms, with SHAPE morph, symmetry, an optional pre-VCA position and level compensation. At 0 it is a true bypass, whatever the symmetry.
- **Drive** with alias-suppressed saturation, analog drift and a per-unit tolerance you can save with the patch.
- **Oversampling** at 1×, 2× or 4× (0, 23 or 26 samples of latency, reported to the host), with a separate setting for offline renders.

Sequencer

- Per-track length from 1 to 32 steps and per-track scale (1/32, 1/16, 1/8T, 1/8) for polymeter.
- Per-track swing and shift. Per-step accent (3 levels), probability, micro-timing, flam (16 types), ratchets (up to 8), pitch bend, and notes and ties for the synth voices.
- Parameter locks: a patch can lock any parameter to its own value on a step (the factory patterns use them). The GRID tab shows the selected step's locks and clears them.
- Clock from the host, the internal clock or the CLK IN jack, picked with the SRC key in the header. In HOST mode SHOGUN locks to your DAW's song position, including after loops and jumps.
- Copy, paste, clear, rotate, randomise, panic and INIT patch.

Modulation

- Four LFOs, each either global or running a separate instance per voice (OWN VOICE), with host sync, sample and hold with slew, and de-clicked shapes.
- A 32-row mod matrix. Sources include the LFOs, each voice's envelope, pitch envelope, velocity, accent and per-hit random, plus note, mod wheel and aftertouch. Any continuous parameter can be a target.

Mixer

- Equal-power pan, mute and solo for every voice.
- Routing to the main mix, four buses (drive, tilt tone, compressor with sidechain) or eight stereo aux outputs.
- Master section with drive, glue compressor, stereo width with mono bass, a tempo-synced delay send, and a soft clipper with an adjustable ceiling.

Patch bay

- 151 jacks: trigger, velocity, pitch, decay, tone, return, output and envelope for every drum. Gate, velocity, note, V/oct, cutoff and note out for each synth voice. Clock and transport in and out, accent out, the four LFOs, random and the mix outputs.

- Jacks follow the Jidai Collection standard: 1 V/oct with C3 at 0 V, ± 5 V signals and colour-coded cable roles. SHOGUN's clock out can drive BUSHIDO directly, and any SHOGUN output can be cabled into RONIN.
- Patches saved with older jack names still load with every cable in place.

Panel

- Eight tabs (MAIN, VOICE, GRID, MOD, SEQ/MIDI, FX/MIX, ROUTE, GLOBAL) on a 1200×672 panel with rack ears, scalable from 75 % to 200 % for HiDPI screens.
- KIT and PATTERN arrows step through INIT and the factory bank, and the search key opens the program list.
- A/B compare (two full snapshots, either copied onto the other) and 64 levels of undo and redo over knob moves, step edits and kit loads.
- Every key on the panel does something: the build checks that no control is left unbound.

MIDI

- Drums on notes 36 to 49 (BD1 to HTC). LEAD on channel 1 and BASS on channel 2, with MIDI note 48 at C3.
- Twelve voice controls on fixed MIDI CCs (the map is on the SEQ/MIDI tab), the mod wheel on CC 1 and channel aftertouch as mod sources, and a velocity curve (linear, soft, hard or fixed).

Install and build

SHOGUN builds from source. You need CMake, a C++17 compiler and JUCE 8.0.4. Set `JUCE_SRC` to your JUCE checkout if it isn't in the default place.

Command	What it does
<code>make plugin</code>	Builds the universal macOS VST3 (Apple silicon and Intel) and signs it for local use
<code>make install-vst</code>	Builds and links <code>SHOGUN.vst3</code> into <code>~/Library/Audio/Plug-Ins/VST3</code>
<code>make plugin-linux</code>	Builds the Linux VST3
<code>make test</code>	Runs the engine test suite
<code>make factory</code>	Rebuilds the factory bank (<code>engine/factory_bank.inc</code>) from <code>scripts/make_factory.py</code> , trimming each kit to its peak target
<code>make web</code>	Compiles the engine to WebAssembly (clang with <code>wasm-ld</code> , plus Node) and writes the playable page <code>web/shogun.html</code>

SHOGUN is built for VST3 hosts such as FL Studio. A new instance starts on the INIT kit with an empty pattern. The factory bank follows INIT: 21 kits, each with its own pattern, from house, techno, acid and electro to breakbeat, jungle, half-time and odd meters. Pick one from your host's program list, with the KIT or PATTERN arrows and displays on the panel, or from the web page's pattern list. A pattern saved on the web page keeps its whole patch document, so step probability, micro-timing, ratchets, parameter locks and mod rows the page has no controls for are kept.

Documentation

- SHOGUN User Manual (PDF): the panel, patching, MIDI, factory kits, DAW setup and the JIDAI RACK back panel
- SCHEMATICS.md: the signal flow and equations for every voice
- TESTPLAN.md: the named tests and the numbers they check
- BUILD_GUIDE.md: engine layout and build notes
- METHODOLOGY.md: the design notes behind the voice choices
- CHANGELOG.md: release history

- docs/SHOGUN_Design_Pack.pdf: the design documents in one PDF, rebuilt with python3 scripts/build_design_pack_pdf.py

The Jidai Collection

SHOGUN uses the same jack names, voltages and cable colours as the rest of the collection, so a cable from one instrument works the same way on another.

- **BUSHIDO**: a 3×12 analog step sequencer with real patch cables. <https://github.com/martialsystems/bushido>
- **RONIN**: a semi-modular synthesizer you patch as an effect. <https://github.com/martialsystems/Ronin>
- **ORIGAMI**: a west-coast triple wave folder, as a standalone effect and as a JIDAI RACK device. <https://github.com/martialsystems/origami>
- **JIDAI RACK**: the collection in one rack, in one plugin. <https://github.com/martialsystems/jidai-collection>

Legal

Copyright © 2026 Martial Systems LLC. All rights reserved. See LICENSE.

SHOGUN is an original Martial Systems design inspired by classic analog drum machines. Martial Systems is not affiliated with or endorsed by any drum machine maker.

Methodology (2026-10-06)

Status

This is a stand-in design for a fan instrument. The voices are analog blocks: oscillators, envelopes, noise, and a clip. Hats and cymbals are square or metallic stacks plus noise. No sample is in the voice model. No coefficient is a measured part value. Each one is marked STAND-IN in SCHEMATICS.md.

No service schematic was found in public, and none is invented here. A block diagram is the farthest the circuit claim goes.

Sources

Read on 2026-10-06.

SHOGUN is inspired by classic analog drum machines. The sources are published owner's documentation for that class of instrument. They are described here by role only. No product, maker, or download location is named.

1. German primary. The German owner's manual: 17 PDF pages, two printed pages on most sheets, printed page numbers through 31, dated 2013 in its PDF metadata. The MIDI appendix is printed page 31 of this file. That appendix is the control-change list used below.
2. English primary. The 17-page English edition of the same manual, with the same contents list (overview through MIDI implementation). Printed pages 3, 4, 6, 8, 9, and 11 were read from an online copy. A separate English PDF file was not downloaded. The contents list and the instrument chapter match the German edition's structure.
3. Published control-change republication on midi.guide, last update 12 September 2024, 64 parameters. This is the list identified by BD1 Attack on CC 2, Decay on 64, Pitch on 65, Tune on 3, Noise on 4, Filter on 5, Dist on 6, and Trigger on 66. Where it disagrees with the 2013 appendix, the appendix wins and the row is marked.

A secondary web conversion of the manual was read only for the Data-versus-Sound conflict named in the brief. It is not a primary.

Later and smaller models of the same family were not used.

Marks

- STAND-IN: a number chosen so the equation is definite. It is not a capture.
- CHOICE: the sources, or the sources plus the brief, support this reading, and the pack uses it.
- ASSUMED: the sources disagree, or they are silent, and a single reading is still required.

Parameter names

Both primary instrument chapters agree on bass drum 1.

English, printed pages 14 to 15: Attack is the level of the attack transients. Decay is volume decay time. Pitch is time and modulation intensity of the pitch envelope. Tune is pitch. Noise is noise level. Filter is the sound of the noise signal. Data is distortion level (printed "Distorion"). Sound selects 1 of 16 different attack transients.

German, printed pages 14 to 15: Data "regelt die Intensität des Verzerrers". Sound: "16 verschiedene Attack-Transienten stehen zur Auswahl". Filter "beeinflusst den Klang des Rauschanteils".

The 2013 appendix names the same jobs as BD1_DIST on CC 6 and BD1_TRIGGER on CC 66. It has no row named Sound.

The secondary conversion says BD1 Data is "Sound of noise signal". That sentence swaps Data with the job both primaries give to Filter.

CHOICE: Sound, which the appendix calls BD1_TRIGGER, selects the transient, index 0 to 15. Data, BD1_DIST, is the clip. Filter stays on the noise.

Quoted disputes

Individual outputs, overview against the jack list

English overview, printed page 4: "Individual outs (in pairs except claps)."

German overview, printed page 4: "Einzelausgänge (bis auf Bassdrum und Clap paarweise ausgeführt)."

Both rear-panel lists then say: BD left is bass drum 1 and BD right is bass drum 2. SD/RS is snare and rim. HH/CY left is open and closed hat, right is cymbal. The clap out is stereo, with the attack transients spread in the field. TO/CO spreads the three toms or congas. CB/CL left is clave, right is cowbell.

The two overview sentences disagree with each other. The jack lists agree with each other and with the brief.

CHOICE: follow the jack list. Maracas, the lead voice, and the bass voice are not on that list. They go to the main mix only.

Cable in an individual out

English printed page 7: "If you plug in a cable on an instrument out, the sound is erased from the main out."

German printed page 7: "Wird ein Kabel in einen Einzelausgang gesteckt, werden die betreffenden Sounds im Hauptausgang gelöscht."

The two primaries agree: the hardware removes those sounds from the main output.

CHOICE for SHOGUN: an individual out is a tap. The voice stays in the main mix. The hardware sentence is recorded so the difference is explicit. TESTPLAN.md locks it as `testIndividualOutStaysInMix`.

Seventeen names, fourteen tracks

Both overviews say 17 drum instruments and 14 tracks that trigger drum instruments. The instrument chapter lists 14 drum names: BD1, BD2, SD, RS, CY, OH, HH, CL, CP, LTC, MTC, HTC, CB, MA. The two note tracks are extra.

ASSUMED: each tom track has a tom mode and a conga mode, and the overview counts those as six names on three tracks. 11 other drums plus 6 is 17 names on 14 tracks. The manuals do not write that sum.

Tom and conga buttons

English, printed pages 15 to 16, gives LTC, MTC, and HTC the same buttons: step button 12 toggles tom and conga, step button 13 enables noise.

German, printed pages 15 to 16: LTC uses the lit LEDs 12 and 13, MTC uses 13 and 14, HTC uses 14 and 15.

The appendix gives each voice its own noise switch and its own tom/conga switch, plus one shared TOM_NOISE on CC 84.

ASSUMED: the voice model uses the per-voice switches and the shared level. The button numbers are a panel procedure, and the two editions do not agree, so they are not part of the equation.

Track length

German overview: "Länge der Spur (1 bis 32 Steps)."

German record chapter, printed page 23: each track may have an individual length "zwischen einem und 16 Steps".

English overview: step number 1 to 32, with four scale settings.

CHOICE: pattern length and per-track length are integers from 1 to 32, which is the overview and the brief. Steps 1 to 16 are the A half. Steps 17 to 32 are the B half. The "1 to 16" sentence matches the sixteen step buttons on the A half.

Scale and length

Both manuals tie scale to a default pattern length: 32nd notes give 32 steps, 16th-note triplets give 24, 16ths give 16, 8th-note triplets give 12. Measure can then set another count. The bar still takes the same time, so a 32nd scale runs the sequencer twice as fast as a 16th scale.

The brief asks for scale as step length, and for patterns of 1 to 32 steps as their own setting.

CHOICE: in SHOGUN, scale sets only how many steps fit in a quarter note. Pattern length is a separate integer from 1 to 32. The hardware coupling is not copied. This split is ASSUMED relative to a machine whose scale button also picked the length.

Cowbell resolution

Both instrument chapters: Data offers 16 tunings, and Sound is the decay time.

Appendix: CB_Tune on CC 85, value 0 to 127, and CB_Decay on CC 86, value 0 to 127.

ASSUMED: Tune and Decay are continuous 0 to 127, under the appendix names. A 16-step control is a quantizer of Tune, step i mapping to $\text{CC round}(i \times 127 / 15)$. The brief's two detuned squares are the oscillator. The manuals do not name the waveform.

Clap type and count

Both instrument chapters: Decay is the tail, Filter is colour, Attack is the level of the attack transients, Data is the number of those transients, Sound is 16 different transients.

The record chapter's hidden-parameter paragraph names BD1, the toms and congas, and the cowbell. It does not name the clap. English printed page 20 and German printed page 20 both still say flam cannot be programmed on Clap, CV1, or CV2/3.

Appendix: CP_DECAY 75, CP_FILTER 18, CP_ATTACK 76, CP_TRIGGER 77. No row is named count, and no row is named Sound.

ASSUMED: CP_TRIGGER is the type, on the same pattern as BD1_TRIGGER. The count is Data, an integer from 1 to 8, and it has no published CC. The record paragraph's shorter instrument list is treated as incomplete next to the instrument chapter.

Set select and one track-delay label

2013 appendix: Set Select, controller 0, value 0 to 2.

midi.guide: Set Select has no controller number, and the range column says 0 to 127.

2013 appendix: Track Delay HH, controller 97.

midi.guide labels controller 97 "Track Track HH". The number matches.

CHOICE: controller 0 with values 0, 1, and 2, and the name Track Delay HH for 97.

Clock bypass

INT and EXT are SHOGUN. They are not a switch in the source manuals. Those manuals have MIDI clock, an analog sync jack that can be an input or an output, and a manual-trigger mode. None of those is this switch.

The switch is the only control that selects the master.

- INT: the internal sequencer triggers the voices. Its tempo is the host tempo while the host is playing, otherwise its own tempo. A cable in a Trig jack does not fire the voice.
- EXT: the internal sequencer does not trigger the voices. Step on and off, flam, and accent from the pattern are ignored for triggering. Knob values still apply. Each voice fires from its own Trig jack. Accent may arrive as velocity on that jack.
- EXT does not stop the clock. The step counter and the clock display keep advancing. Switching back to INT connects the triggers to the step the clock has reached.
- A cable in a Trig jack does not force EXT.

ASSUMED: "resumes" means the counter never froze. The alternative, a frozen counter that continues from the step where EXT was engaged, would stop the clock the brief says keeps running.

Voices the manuals do name

The instrument chapter gives the jobs below. Ranges and waveforms that the chapter does not give are in SCHEMATICS.md, marked there.

- BD1: attack level, decay, pitch envelope time and depth on one knob, tune, noise level, noise filter, distortion, and 1 of 16 attack transients.
- BD2: decay up to a steady tone, tune, and tone as the attack level. The chapter does not give BD2 a noise, a filter, a distortion, or 16 transient types. It is a second kick.
- SD: tune shared by two tones, detune of tone 2, snappy noise level, noise decay, blend of the two tones, decay of both tones, and pitch-envelope intensity.
- RS: pitch only. The brief asks for a short pitched click.
- CY: decay, blend of both signals, and pitch or colour.
- OH and HH: separate decays, and one pitch or colour shared by both. The appendix exposes that colour as HH_TUNE on CC 73. There is no OH tune row.
- CL: tune and decay.
- CP: tail decay, filter, attack level, a count of attack transients, and 16 transient types. The clap out spreads those transients. The brief adds the spacing, about 10 to 12 ms, which the manuals do not print.
- LTC, MTC, HTC: tune, decay up to a steady tone, noise on or off, tom or conga, and one noise level for all three. Bend is available. The tom out spreads the three.
- CB: tune and decay in the appendix. The brief specifies two detuned squares.

- MA: decay only. The brief specifies filtered noise.
- Lead and bass: the overview says one parameter each. Bass Data is "Filter cutoff or CV 3 value" in the English chapter. Setup plays a steady 440 Hz and tunes both voices with Data. The waveform is not named. ASSUMED waveform: a band-limited saw for each. ASSUMED lead parameter: a one-pole tone, because the chapter names one parameter and does not name the lead knob.

Closed hat kills open hat. The pages read do not state a choke time. The brief requires the choke, so the pack defines it: the closed-hat trigger clears the open-hat envelope on that sample.

Sequencer the manuals do name

- Patterns live in 3 sets of 3 banks of 16, which is 144. That memory map is the hardware's. SHOGUN stores patterns without a 144-slot cap in this pack. The set-select controller is still defined.
- A and B. A pattern longer than 16 steps has a B half, and the machine walks it. A/B can also alternate. Copy of A onto B exists on the hardware. Chain is runtime only: the English overview calls it a fill that is not storable, and the German overview says the chain is "nicht speicherbar".
- Per track: length, shuffle, shift, mute. Shuffle in record mode is per track, 16 intensities. Shuffle in play mode is global and does not replace the stored per-track values when it is released. Shift is controllers 89 to 104, described as a small delay of the whole track. No millisecond range is printed.
- Per step, drums: on or off, one of three accent levels (off or soft, green or medium, red or loud), flam on 16 patterns, bend where the instrument has it, and an extra sound value where the instrument chapter has one. Bend instruments are BD1, BD2, SD, LTC, MTC, and HTC. Flam is unavailable on clap and on both note tracks.
- Per step, note tracks: on or off, pitch over three octaves, accent on the bass track, and a second value on the bass track that is CV3 and the internal bass cutoff. A/B enters a silent step. Select ties steps into a longer note.
- Level pots and the master volume are not stored in the pattern.
- Manual record mode shares one knob set across steps. Accents and flams can still differ per step. Step and jam modes can store different knob values per step.

Tempo

English play chapter: the tempo knob covers approximately 60 to 180 BPM, and there is no tempo readout.

CHOICE: SHOGUN's internal tempo uses those endpoints as exact bounds, default 120. The clock display is part of SHOGUN. The hardware has no readout.

Gate level

A Trig jack is a rising edge through 1.0 V on a gate that rests at 0 V and is high at 5 V. That 1 V threshold is the collection's existing gate rule between RONIN and BUSHIDO. Velocity, when the sender has one, is an integer 0 to 127 riding with the edge. A plain gate with no velocity uses 127.

ASSUMED: the plain-gate default of 127. The source manuals say drum notes carry velocity. They do not define a jack that has no velocity byte.

Schematics (2026-10-06)

Functional blocks and sample equations. Coefficients are STAND-IN. Sample rate in every numeric example: 48,000 Hz. Arithmetic is IEEE-754 double. A later test may treat the printed decimals as exact within 1e-5.

Shared helpers:

$u(cc) = cc / 127$

```

fs = 48000
pi = 3.141592653589793

```

cc is an integer from 0 to 127. A per-step override, when the record mode stored one, replaces the live knob for that step. Manual record mode does not store per-step knobs.

A rising trigger at sample n0 resets that voice's phase to 0 and restarts its envelopes at n0. Until the first trigger, a cleared voice outputs 0.

Decay law. Every drum Decay knob, and the snare's Tone Decay and SN Decay, sets its time constant the same way. STAND-IN:

```

decay_tau(u) = 0.008 * exp(4.5 * u)      # seconds

```

u = 0 is 8 ms, u = 0.5 is 75.902 ms (CC 64 is 77.259 ms), and u = 1 is 720.137 ms. The clap tail uses it. The clap bursts keep their fixed 3 ms. RS has no Decay knob. BD2 at Decay 127 is the one exception: it holds. A tom at Decay 127 rings for 4 s and then ends.

A drum voice ends when every envelope it has is under 1e-3. From that sample, that sample included, it outputs 0 until its next trigger. Only a held BD2 does not end, because its envelope stays at the sustain.

Rests. A drum step that is off fires nothing, and a drum voice that is still ringing rings out. A note-track rest releases the note that is sounding, with the release in Lead and bass.

Velocity law

Pattern accent index a is 0, 1, or 2. English printed page 20: LED off is soft, green is medium, red is loud.

```

g_accent(0) = 0.55
g_accent(1) = 0.78
g_accent(2) = 1.00

```

STAND-IN.

EXT velocity v, an integer 0 to 127:

```

g_vel(v) = 0.15 + 0.85 * (v / 127)

```

STAND-IN. v = 0 gives 0.150000. v = 100 gives 0.819291. v = 127 gives 1.000000.

INT multiplies the voice by g_accent and ignores Trig velocity. EXT multiplies by g_vel and ignores the pattern accent. The gain multiplies the voice output after the voice equation.

Clock

Steps per quarter note, from the four scale settings in both manuals:

Scale	Steps per quarter	Samples per step at 120 BPM
32nd	8	3,000
16th triplet	6	4,000
16th	4	6,000
8th triplet	3	8,000

```

period = fs * 60 / (bpm * steps_per_quarter)

```

At 120 BPM and a 16th scale, period is 6,000 samples. Internal bpm is clamped to 60 through 180. Default 120.

The step counter increments on each period boundary in INT and in EXT. The clock display shows (counter mod pattern_length) + 1. pattern_length is the bar, an integer from 1 to 32.

Each track stores its own length L, also 1 to 32. On global counter c the track reads step c mod L. A shorter track cycles against the bar. A longer track has steps past one bar, which is how a track reaches the B half when the bar is shorter. The track fires when that stored step is on. It does not also have to fall inside the bar length.

Shuffle intensity s is an integer 0 to 15. STAND-IN. The delay goes on odd clock steps, counter c odd, whatever the track's length. A track of odd length still swings on the beat, so its own step 0 is delayed on every other pass:

$$\text{delay_samples} = (s / 15) * (\text{period} / 3)$$

$s = 0$ is no delay. $s = 8$ at period 6,000 delays an odd clock step by 1,066.666667 samples. The fractional sample is kept and the trigger fires on the sample that contains the fractional boundary. Play-mode global shuffle, while held, uses one s for every track and does not write the pattern.

Track shift, controllers 89 to 104, one per track including the two note tracks. STAND-IN:

$$\text{shift_samples} = \text{round}(u(cc) * 0.030 * fs)$$

Full scale is 1,440 samples, 30 ms. The whole track's triggers move later by that many samples.

INT and EXT

INT fires a drum voice when its step is on. The flam table and the accent index apply. The Trig jack is not read.

EXT does not fire from the pattern. Flam, step on and off, and pattern accent do nothing. The Trig jack's rising edge fires the voice, and g_vel supplies accent. Knobs, including per-step knob overrides if a later editor still wants them, remain in force. This pack's EXT path uses the live knobs, not the pattern's per-step sound overrides, because the brief says the pattern is ignored for step on and off, flam, and accent, and that knob values still apply. ASSUMED: "knob values" means the live knobs, not values stored on ignored steps.

Lead and bass in EXT follow the live gate on their Trig jacks. A rising edge opens the note at the live pitch with g_vel . The falling edge starts the release. The pattern's notes, rests, and ties do nothing in EXT.

Bend stored on a step is part of the pattern. EXT ignores it. The Pitch knob on BD1, BD2, SD, and the toms still applies, because that knob is a sound control, not the step bend.

Clock jacks

CLK IN, RST IN, and RUN IN are gate inputs at the same 1 V threshold as the Trig jacks. A rising edge on CLK IN advances the counter one step, and while CLK IN is patched the internal period does not. A rising edge on RST IN sets the counter to 0, so the next step is step 1. A rising edge on RUN IN starts a stopped clock or stops a running one. None of them changes the INT and EXT switch. Only the switch chooses.

Noise generator

One generator for the whole instrument, so examples reproduce.

$$\begin{aligned} \text{state} &= (1664525 * \text{state} + 1013904223) \bmod 2^{32} \\ x &= \text{state} / 2^{32} * 2 - 1 \end{aligned}$$

Initial state is 1. The first draw yields $x = -0.527089$. Each noise consumer takes the next draw when it needs a sample. The maracas example below is the first draw of a cleared generator.

One-pole lowpass, state y previous 0 at a trigger:

$$\begin{aligned} a &= \exp(-2 * \pi * fc / fs) \\ y[n] &= (1 - a) * x[n] + a * y[n-1] \end{aligned}$$

Output sum

Instrument level and master are linear gains, not stored in the pattern. Default level 1, default master 1. STAND-IN, they are plain multipliers.

Equal-power pan, p from -1 to 1:

$$\begin{aligned} g_L &= \cos(\pi / 4 * (1 + p)) \\ g_R &= \sin(\pi / 4 * (1 + p)) \end{aligned}$$

STAND-IN. Every centred voice, $p = 0$, is 0.707107 on each side, which is $\sqrt{0.5}$, so it sits level with a hard-panned voice. $p = -0.7$ is $g_L = 0.972370$, $g_R = 0.233445$.

Voice	Pair	p
BD1	BD L	hard left, written only to L at gain 1
BD2	BD R	hard right, written only to R at gain 1
SD	SD/RS L	hard left
RS	SD/RS R	hard right
OH + HH	HH/CY L	summed, hard left
CY	HH/CY R	hard right
CP bursts	CP stereo	pan of burst i, below
CP tail	CP stereo	$p = 0$
LTC	TO/CO	$p = -0.7$
MTC	TO/CO	$p = 0$
HTC	TO/CO	$p = 0.7$
CL	CB/CL L	hard left
CB	CB/CL R	hard right
MA, lead, bass	main only	$p = 0$

The voice sample is after the accent or velocity gain and after the instrument level. Hard left adds that sample to the left channel of the pair and of the main, and adds 0 to the right. Hard right is the swap. A panned voice uses g_L and g_R on both the pair and the main. The master multiplies the main only. Pair jacks are before the master.

Solo. One voice at a time can be soloed, or none. While a voice is soloed, only that voice reaches its pair and the main. Every other voice still triggers and runs its envelopes, and outputs 0, so when solo is turned off each one is heard where it would have been. A muted track stays silent when it is soloed: mute wins. Solo is not stored in the pattern, and it does not change the INT and EXT switch.

CHOICE: the main mix is that sum whether or not a pair jack is patched. Maracas, lead, and bass have no pair.

Flam table

Flam exists on every drum track except clap. Sixteen patterns, index k from 0 to 15. The manuals name 16 patterns and do not print the timings. STAND-IN offsets, in samples, from the step:

```
hits = 2 + (k mod 4)
gap = 180 * (1 + floor(k / 4))
offsets = 0, gap, 2*gap, ... while the list length is hits
```

Index 0 is samples 0 and 180. Index 15 is 0, 720, 1,440, 2,160, and 2,880. Each offset fires the voice again. A new fire restarts that voice's envelopes. Offsets may cross into the next step.

Wave

BD1, BD2, LTC, MTC, and HTC each have a Wave knob. Hats, clap, cymbal, and maracas do not. Wave is the west-coast triple wave folder: six identical cells in series, ported from martialsystems/serge_middle (the cell, its Lambert W solver, and its drive peak table), not a second waveshaper.

Each cell, for v not 0, with $VT = 0.02585$, $Is = 2.52e-9$, $\eta = 1.68$, $R = 33,000$:

```
z = (Is * R) / (eta * VT) * exp(|v| / (eta * VT))
v_out = sign(v) * (|v| - 2 * eta * VT * W(z)) # v_out(0) = 0
C6(v) = C(C(C(C(C(C(v)))))))
```

W is the principal Lambert W , solved by Halley on $w + \ln(w) - \ln(z) = 0$ so the exponential is never formed. The residual is under $1e-12$ through 6 V and at 48 V.

```
g(cc) = 0 at CC 0, the bypass
```

```

g(cc)    = 0.5 + 3.5 * (cc - 1) / 126    CC 1 to 127, 0.5 to 4. STAND-IN taper
fold(x)   = x                            if wave_cc = 0
fold(x)   = (4.379272 * P(1) / P(g)) * C6(5 * g * x) otherwise

```

x is the body oscillator, sin(phase), about +/-1, scaled to +/-5 V times g before the cells. P(g) is the serge_middle peak of |C6(v)| for |v| <= 5 g, linear between its 53 knots, so a full-scale x peaks at 1 after the cells at every g and Wave does not change the peak level. Default Wave is CC 32 (0.25), g = 1.361111.

The fold acts on the oscillator, before the body envelope, so the decay law and the 1e-3 end are untouched. On BD1 the Dist clip comes after Wave. The 4x oversampler, the 10 Hz DC block, and the knob smoother of serge_middle are not in this pass. A folded sine is odd, so it has no DC.

Measured on a held body at velocity 1, the share of spectrum energy above 1.5 times the fundamental is under 0.005 at Wave 0, 0.388 at the default, and 0.033 at CC 127. The folds sit in the bottom 5.2 V of the cell's input (fold roots 0.73, 1.55, 2.43, 3.34, 4.27, 5.22 V). Past that each cell is close to an inverter, so at g = 4 most of the 20 V swing passes through and, after the peak is brought back to 1, the folds are a ripple near each zero crossing. The densest setting is the low end of the knob.

The init kit at the default Wave: one BD1 hit peaks at 0.963 against 0.964 at Wave 0, and its RMS over 0.5 s is 0.106 against 0.155 (3.3 dB lower).

BD1

```

trig --> pitch and bend envelopes --> sine --> Wave --+
      --> click[s], 1 ms -----+--> sum --> tanh clip --> gain
      --> noise burst on the body env --> one-pole --+

```

Density pass, STAND-IN coefficients set by ear, no hardware capture. The clip is bypassed at Dist 0.

Decay does not reach a steady tone. The body envelope falls to 0 for every knob value.

```

f_tune = 35 * (140/35) ^ u(tune)          # 35 Hz to 140 Hz
depth  = 18 * u(pitch)                    # semitones, 0 to 18
tau_p  = 0.012 + 0.25 * u(pitch)          # seconds
tau_bend = max(tau_p, 0.08)               # the step bend's own time, 80 ms floor
tau_b  = decay_tau(u(decay))              # 8 ms to 720 ms
s      = min(15, floor(trigger_cc * 16 / 128))
f_tr   = 160 * (1.35 ^ s)
fc     = 200 * (8000/200) ^ u(filter)     # 200 Hz to 8,000 Hz
drive  = 9 * u(dist)                      # 0 at Dist 0, which is the bypass

```

```

shape(ph, k) = tanh(k * sin(ph)) / tanh(k)

```

```

p_env[n] = exp(-n / (fs * tau_p))
b_env[n] = exp(-n / (fs * tau_bend))      # 0 for every n when bend_st = 0
f[n]     = f_tune * 2 ^ ((depth * p_env[n] + bend_st * b_env[n]) / 12)
body[n]  = fold(sin(phase[n])) * exp(-n / (fs * tau_b))      # Wave, above
click[n] = wave_s(n) * (1 - n / 48) ^ 2   for n < 48, else 0
burst[n] = x[n] * exp(-n / (fs * tau_b)) * exp(-n / (fs * 0.015))
noise    = u(noise_cc) * lowpass(burst[n], fc)
pre[n]   = body[n] + u(attack) * click[n] + noise
y[n]     = pre[n]                          if dist_cc = 0
y[n]     = tanh(drive * pre[n]) / tanh(drive)  if dist_cc > 0

```

The click is 1 ms, one of 16 shapes. s mod 4 picks the wave and s sets its rate f_tr: 0 is sin(2 pi f_tr n / fs), 1 is tanh(6 sin(2 pi f_tr n / fs)) / tanh(6), 2 is cos(2 pi f_tr n / fs), a hard tick, and 3 is a saw, 2 * frac(f_tr * n / fs) - 1. STAND-IN.

The noise is a short burst under the body envelope, then the one-pole. The burst is 0 wherever the body is, so the voice is silent once the body is under 1e-3. There is no separate noise decay knob.

phase advances by $2 * \pi * f[n] / fs$ after the sample is taken. n is samples since the trigger. bend_st is the step bend in semitones. Like the snare, it is its own drop to Tune: its time is the Pitch time, but never under 80 ms while a bend is set. INT step bend is bend_st = 12 * (2 * u(bend_cc) - 1). CC 0 is -12 semitones, CC 127 is +12, and CC 64 is 12 * (128/127 - 1) = 0.094488 semitones. STAND-IN. EXT forces bend_st to 0. The printed BD1 example uses bend_st = 0, noise CC 0, dist CC 0, and the default Wave, CC 32, so y = pre. drive = 9u starts at 0.070866 for CC 1, where tanh(drive * pre) / tanh(drive)

is within 0.2% of pre, so CC 0 to CC 1 does not jump.

Example knobs: Attack 64, Decay 80, Pitch 40, Tune 50, Noise 0, Filter 64, Dist 0, Trigger 0.

```
u(attack) = 0.503937
f_tune    = 60.408707 Hz
depth     = 5.669291 semitones
tau_p     = 0.090740 s
tau_b     = 0.136195 s
s         = 0
f_tr      = 160 Hz
```

n	f Hz	y
0	83.814360	0.000000
10	83.751440	0.055968
48	83.514083	0.040808
480	80.998689	-0.482608
2,400	72.957296	0.044642

The instantaneous frequency at n = 2,400 is closer to f_tune than the frequency at n = 0. That is the pitch envelope decaying. The small y values are the fold: at n = 48 sin(phase) is about 0.5, which is 3.4 V into the cells, close to a fold root.

Same knobs, Dist 64: drive = 4.535433, and at n = 48 y = 0.183040.

Same knobs, Attack 127, sample n = 10: Trigger 0 gives s = 0, a sine click at f_tr = 160, y = 0.120608. Trigger 64 gives s = 8, a sine click at f_tr = 1,765.184603, y = 0.453201. The attack sample changed because the click rate changed.

BD2

```
trig --> sine with slow FM --> Wave, envelope toward sustain or toward 0 --> out
--> octave sine, 5 ms, level = tone ----->
```

No noise, no filter, no clip, no 16-type transient. The body can hold.

```
f_tune = 45 * (100/45) ^ u(tune)      # 45 Hz to 100 Hz
f_tr    = 2 * f_tune
tau_tr  = 0.005
f_m     = 40 * (200/40) ^ u(tone)     # FM rate, 40 Hz to 200 Hz
i       = 0.12 * f_tune               # FM depth in Hz, STAND-IN

if decay_cc >= 127:
    sustain = 0.70
    tau_b   = 0.40
else:
    sustain = 0.0
    tau_b   = decay_tau(u(decay))

env[n] = sustain + (1 - sustain) * exp(-n / (fs * tau_b))
f[n]   = f_tune * 2 ^ (bend_st * exp(-n / (fs * 0.08)) / 12) + i * sin(2 * pi * f_m * n / fs)
body[n] = fold(sin(phase[n])) * env[n]      # BD2 Wave
tr[n]   = sin(2 * pi * f_tr * n / fs) * exp(-n / (fs * tau_tr))
y[n]    = body[n] + u(tone) * tr[n]
```

fold is the Wave folder on BD2's own knob. The slow FM lets it bark: the pitch swings f_tune by 12% at the Tone rate, and it keeps swinging while Decay 127 holds. Tone sets both the FM rate and the octave transient level.

Pitch bend, when the step has one, uses tau_p = 0.08 s STAND-IN and depth_st = 12 * (2 * u(bend_cc) - 1). The Tune knob is the resting frequency. The example below has no bend.

Hold check, envelope only: Decay 127 at n = 0 is 1.000000, at n = 480 is 0.992593, at n = 96,000 (2 s) is 0.702021, against sustain 0.700000. Decay 0 at n = 96,000 is 0.000000 within 1e-5. Decay 126 at n = 96,000 is 0.056280 and is not a hold. Only CC 127 holds.

Tone example, Tune 60, Tone 100, n = 10, envelopes of the transient only: $f_{\text{tune}} = 65.621948$ Hz, $f_{\text{tr}} = 131.243897$ Hz, $u(\text{tone}) * \text{tr} = 0.129116$, $f_{\text{m}} = 141.244575$ Hz, and f at $n = 10$ is 67.077722 Hz.

SD

```

trig --> pitch envelope --> shaped tone 1 --+
      --> shaped tone 2 --+--> blend --> out
      --> noise --> 4-pole low-pass, ducked --> noise envelope -->

f1      = 120 * (400/120) ^ u(tune)      # 120 Hz to 400 Hz
detune_st = -8 + 16 * u(d_tune)          # -8 to +8 semitones
f2      = f1 * 2 ^ (detune_st / 12)
depth   = 14 * u(pitch)                  # semitones
tau_p   = 0.01 + 0.12 * u(pitch)
tau_tone = decay_tau(u(tone_decay))
tau_n   = decay_tau(u(sn_decay))
tau_bend = max(tau_p, 0.08)              # the bend's own time, 80 ms floor

p_env[n] = exp(-n / (fs * tau_p))
b_env[n] = exp(-n / (fs * tau_bend))      # 0 for every n when bend_st = 0
f1[n] = f1 * 2 ^ ((depth * p_env[n] + bend_st * b_env[n]) / 12)
f2[n] = f2 * 2 ^ ((depth * p_env[n] + bend_st * b_env[n]) / 12)
fc_n   = 1000 * (12000/1000) ^ u(tone)    # noise low-pass, 1 kHz to 12 kHz
t1[n] = shape(phase1, 2) * exp(-n / (fs * tau_tone))
t2[n] = shape(phase2, 2) * exp(-n / (fs * tau_tone))
lp[n] = ladder(duck[n] * x[n], fc_n)
nz[n] = u(snappy) * lp[n] * exp(-n / (fs * tau_n))
y[n] = (1 - u(tone)) * t1[n] + u(tone) * t2[n] + nz[n]

```

Example, noise left out (Snappy 0): Tune 70, D-Tune 90, Snappy 0, SN Decay 50, Tone 64, Tone Decay 60, Pitch 30.

```

f1      = 233.014061 Hz
f2      = 282.574688 Hz
detune_st = 3.338583 semitones
tau_tone = 0.067049 s
blend    = 0.503937

```

$\text{shape}(ph, k) = \tanh(k * \sin(ph)) / \tanh(k)$, a sine bent toward square, here at a fixed $k = 2$, STAND-IN. SD has no Wave knob. The noise goes through a 4-pole low-pass, four one-poles at $g = 1 - \exp(-2 \pi f_{c_n} / fs)$ with feedback 1.6 from the last pole, output times 2.6 so the pass band is unity. Tone sets its cutoff, so Tone 0 is dark. Resonance can pump: a peak follower on $|p|$ (instant attack, 10 ms release) ducks the noise input, $\text{duck} = 1 - 0.25 * \min(1, \text{follow})$. It is a stand-in compressor, not a real one. Snappy is the noise level, on the short decay curve.

At $n = 20$, $t1 = 0.899876$, $t2 = 0.943686$, $y = 0.921953$. Both tones are in the sum. Tone CC 0 would output $t1$. Tone CC 127 would output $t2$.

Step bend on SD is a pitch drop on top of Tune with its own envelope. It is not a deeper Pitch knob, and there is no second oscillator. While bend_st is not 0 its time is the Pitch time, but never under 80 ms, so Pitch 0 with a bend still swoops. $\text{bend_st} = 0$ adds nothing, and the floor does nothing. Tune 70, Pitch 0, $\text{bend_st} = +12$: $f1[0] = 466.028122$ Hz, and at $n = 3,840$ (80 ms) $f1 = 300.694079$ Hz. On the Pitch time alone (10 ms at Pitch 0) it would already be 233.068249 Hz, the same as Tune within 0.06 Hz. Pitch 127 and $\text{bend_st} = +12$: tau_bend is $\text{tau_p} = 0.13$ s, and at $n = 6,240$ $f1 = 404.878529$ Hz.

RS

```

trig --> short sine --> out

f  = 250 * (2500/250) ^ u(tune)          # 250 Hz to 2,500 Hz
tau = 0.012                              # fixed, no decay knob
y[n] = sin(2 * pi * f * n / fs) * exp(-n / (fs * tau))

```

Tune 40: $f = 516.298233$ Hz. At $n = 8$, $y = 0.507608$.

CY

```
trig --> six squares, inharmonic --+--> mix --> band-pass --> decay --> out
--> noise -----+
```

The metal is six band-limited squares (PolyBLEP) at inharmonic ratios, averaged. No sample. STAND-IN ratios, mix, band, and makeup:

```
ratios = 1, 1.4471, 1.6170, 1.9265, 2.5028, 2.6637
f0      = 180 * (900/180) ^ u(tune)          # 180 Hz to 900 Hz
mix      = 0.1 + 0.8 * u(tone)                # noise share, metal against noise
tau      = decay_tau(u(decay))

metal[n] = mean over r of square(f0 * r, n)
y[n]      = 2.0 * bandpass((1 - mix) * metal[n] + mix * x[n], 5,500 Hz, Q 0.8) * exp(-n / (fs * tau))
```

Each square starts at phase 0 on the trigger. The band-pass is the RBJ constant 0 dB peak form. Tune moves the stack, not the noise level. Tone is the mix.

Tune 64: f0 = 405.050673 Hz. Tone 70: mix = 0.540945.

OH and HH

One colour, two decays, one choke.

```
f0      = 250 * (1000/250) ^ u(hh_tune)      # 250 Hz to 1,000 Hz, CC 73
tau_oh   = decay_tau(u(oh_decay))
tau_hh   = decay_tau(u(hh_decay))
y[n]     = 2.5 * bandpass(0.65 * metal[n] + 0.35 * x[n], 8,000 Hz, Q 1.0) * exp(-n / (fs * tau))
```

Each hat is the CY six-square metal at f0, mixed with noise at a fixed 0.35 share, through a band-pass at 8 kHz, times its own exp decay. STAND-IN mix, band, and makeup. Tune moves the stack, not the noise level. No hat filter knob.

A closed-hat trigger sets the open-hat envelope to 0 on that sample, and the open hat stays at 0 until the next open trigger. It does not shorten the closed hat.

Example colour, HH Tune 60: f0 = 481.257820 Hz. OH Decay 100: tau_oh = 0.276649 s. HH Decay 40: tau_hh = 0.033008 s. An open hat that has run 200 samples has envelope 0.985052. On the closed trigger that envelope is 0.

CL

```
f      = 400 * (3000/400) ^ u(tune)          # 400 Hz to 3,000 Hz
tau     = decay_tau(u(decay))
y[n]    = sin(2 * pi * f * n / fs) * exp(-n / (fs * tau))
```

Tune 50, Decay 30: f = 884.244363 Hz, tau = 0.023160 s. At n = 12, y = 0.972955.

CP

```
trig --> noise --> high-pass --> N bursts, 3 ms, 10 to 12 ms apart, panned --> stereo
--> one-pole low-pass, delayed tail -----> both channels
```

```
s      = min(15, floor(trigger_cc * 16 / 128)) # CC 77, ASSUMED type
count  = 1 + floor(data_cc * 8 / 128)         # 1 to 8, ASSUMED, no CC
f_hp   = 300 * (1.2 ^ s)                       # 300 Hz to 4,622 Hz
gaps   = 480, 576, 528, 504, 552, 576, 480    # samples, 10 ms to 12 ms
tau_tr = 0.003                                 # fixed, Decay never stretches a burst
fc     = 400 * (6000/400) ^ u(filter)
tau     = decay_tau(u(decay))

hp[n]   = x[n] - lowpass(x[n], f_hp)
burst_i[n] = hp[n] * exp(-n_b / (fs * tau_tr)) # n_b = n - start_i, 0 before the start
tail[n]  = lowpass(hp[n], fc) * exp(-(n - t_0) / (fs * tau)) for n >= t_0, else 0
t_0     = start_(count-1) + 528
```

Burst i starts at the sum of the first i gaps: 0, 480, 1,056, 1,584, 2,088, 2,640, 3,216, 3,696. Each burst is the same high-passed noise under its own fixed 3 ms decay. Pan of burst i , $\text{count} > 1$: $p = -1 + 2 * i / (\text{count} - 1)$. One burst uses $p = 0$. The tail is one filtered delay: the same high-passed noise through the one-pole at f_c , opening one gap (11 ms) after the last burst starts and decaying on the drum law, times 0.707107 on each channel. The one-pole runs from the hit, so it is settled when the tail opens. STAND-IN.

```
yL[n] = u(attack) * sum_i burst_i[n] * gL(p_i) + tail[n] * 0.707107
yR[n] = u(attack) * sum_i burst_i[n] * gR(p_i) + tail[n] * 0.707107
```

Data CC 48 gives count 4. At any one sample, burst i over burst $i - 1$ is $\exp(\text{gap} / 144)$: the bursts share one noise and one 3 ms curve.

LTC, MTC, HTC

Each voice:

```
trig --> sine with slow FM --> Wave, ring --> out
--> noise, if that voice's switch is on, shared level
```

Tom mode is the sine through that tom's own Wave folder, with the BD2 FM term at a quieter depth and a fixed rate: $f[n] = f_{\text{tune}} * \text{bend} + 0.04 * f_{\text{tune}} * \sin(2 * \pi * 50 * n / fs)$. STAND-IN. Conga mode adds a second sine at $2.30 * f_{\text{tune}}$, gain 0.35. ASSUMED. The manuals say the switch changes tom and conga and do not describe the waveform.

```
LTC f = 70 * (180/70) ^ u(tune)
MTC f = 100 * (280/100) ^ u(tune)
HTC f = 140 * (400/140) ^ u(tune)
```

Below CC 127 Decay is the shared decay law. At CC 127 a tom rings and then ends. It does not hold, because a tom that never ends is a stuck voice. Only BD2 drones. STAND-IN:

```
if decay_cc >= 127:
    sustain = 0.55
    tau_b = 0.35
    ring[n] = sustain + (1 - sustain) * exp(-min(n, 192000) / (fs * tau_b))
    env[n] = ring[n] for n <= 192,000 (4 s)
    env[n] = ring[192000] * exp(-(n - 192000) / (fs * 0.05)) after that
else:
    env[n] = exp(-n / (fs * decay_tau(u(decay))))
```

There is no choke jack. The next hit replaces the ring, as for every drum voice.

Noise, when the voice switch is on (CC at or above 64): $u(\text{tom_noise}) * x[n] * \exp(-n / (fs * 0.12))$. The 0.12 s noise decay is STAND-IN and shared. tom_noise is CC 84.

LTC Tune 40: $f_{\text{tune}} = 94.251191$ Hz ($f[0]$, before the FM moves). Decay 127 at $n = 96,000$: envelope 0.551484, sustain 0.550000. At $n = 192,000$ (4 s) it is 0.550005. 50 ms into the release, at $n = 194,400$, it is 0.202335. It is under $1e-3$ at $n = 207,144$ (4.32 s), and the tom ends there. TOM_NOISE 33: level 0.259843. On the first noise draw that is -0.136958, and on the TO/CO left channel at $p = -0.7$ it is -0.133176.

Bend uses the BD2 bend paragraph.

CB

```
trig --> square at f ----+
--> square at 1.015 f --> average --> decay --> out
```

The two squares are the brief. Detune ratio 1.015 is STAND-IN. A square is the odd harmonics $k = 1, 3, 5, \dots$ while $k * f$ is under Nyquist and $k \leq 15$, amplitude $1/k$, divided by the sum of those amplitudes.

```
f = 300 * (1200/300) ^ u(tune) # 300 Hz to 1,200 Hz
tau = decay_tau(u(decay))
y[n] = 0.5 * (square(f) + square(1.015 * f)) * exp(-n / (fs * tau))
```

Tune 48, Decay 70: $f = 506.607352$ Hz, second square 514.206462 Hz, $\tau = 0.095560$ s. At $n = 15$, $y = 0.404500$.

Sixteen panel steps, if a UI wants them, are Tune CC values $\text{round}(i * 127 / 15)$ for i from 0 to 15. The voice reads the continuous CC.

MA

```
trig --> noise --> one-pole lowpass at 1,500 Hz --> decay --> main
```

No tune knob. Colour is fixed. STAND-IN $f_c = 1,500$ Hz. The filter is the lowpass in the shared section, held at that cutoff.

Decay:

```
tau = decay_tau(u(decay))
y[n] = lowpass(x[n]) * exp(-n / (fs * tau))
```

Decay 55: $\tau = 0.056163$ s. First noise draw $x = -0.527089$. $a = \exp(-2 * \pi * 1500 / fs) = 0.821725$. With filter state 0, $y[0] = -0.093967$ before the envelope, and the envelope at $n = 0$ is 1. On the main, at the equal-power centre, each side is -0.066445 .

Lead and bass

```
note --> band-limited saw --> one-pole lowpass --> envelope --> main
```

Waveform ASSUMED: saw, harmonics $k = 1$ to 8, amplitude $1/k$, skipped when $k * f$ is at or above Nyquist, then divided by the sum of the weights used.

Pitch is MIDI note 36 to 72, which is the appendix range for CV1 and CV23. $\text{Hz} = 440 * 2^{((\text{note} - 69) / 12)}$. Note 60 is 261.625565 Hz. Note 36 is 65.406391 Hz. Note 72 is 523.251131 Hz.

The panel entry in both manuals is step buttons 1 to 13 for pitch class C through the next C, and buttons 14 to 16 for the octave. ASSUMED map onto notes 36 to 72: octave 0, 1, 2 from buttons 14, 15, 16, pitch class 0 to 12 from buttons 1 to 13, $\text{note} = 36 + 12 * \text{octave} + \text{class}$, then clamp to 72. A class of 12 is the upper C, so some combinations land on the same note. That collision is accepted.

A rest, the A/B silent step, releases the note that is sounding. It does not cut it to 0. A rest on a voice that is not sounding opens nothing. A tied note keeps the envelope open and does not retrigger.

Envelope: attack is one sample to 1 on a new note, hold while the note is tied, release $\tau = 0.03$ s STAND-IN when the note ends.

Lead tone, ASSUMED, one-pole lowpass:

```
fc = 200 * (8000/200) ^ u(lead_tone)
```

Bass tone is the manual's cutoff, and the same u is the CV3 value, scaled STAND-IN as 0 V to 5 V for a jack: $\text{cv3} = 5 * u(\text{bass_tone})$.

```
fc = 80 * (4000/80) ^ u(bass_tone)
```

Lead example, note 60, before the filter, $n = 25$ samples after a trigger, harmonics 1 to 8: $\text{saw} = 0.387776$. Lead tone CC 80: $f_c = 2,042.686148$ Hz.

Bass accent uses g_accent on the bass voice in INT. The drums' three-level accent is the same table.

Step bend and the Pitch knob

BD1's Pitch knob is both time and depth, one u , as the instrument chapter states. SD's pitch control is depth, with the time law above. Step bend is an extra signed depth on BD1, BD2, SD, LTC, MTC, and HTC only. On SD it has its own envelope with the 80 ms floor, written in the SD equation. CY, RS, hats, clave, clap, cowbell, and maracas have no step bend. EXT ignores step bend.

Init kit (INIT)

STAND-IN. A balance set by ear from the knob ranges above, with no samples. A new engine loads it. `reset()` still clears to empty knobs, empty steps, level 1, and master 1, which is the state every named test arms from. The plugin writes its own parameters over both on the first block.

Knob CC is $\text{round}(u * 127)$.

Voice	Knobs, u (CC)	Level
BD1	Tune 0.22 (28), Pitch 0.55 (70), Decay 0.28 (36), Attack 0.45 (57), Dist 0.15 (19), Noise 0.08 (10), Filter 0.35 (44)	0.85
BD2	off	0
SD	Tune 0.48 (61), Detune 0.12 (15), Pitch 0.35 (44), Tone Decay 0.22 (28), SN Decay 0.30 (38), Snappy 0.62 (79), Tone 0.45 (57)	0.75
RS	Tune 0.62 (79)	0.4
CP	Data 48 (4 bursts), Attack 0.7 (89), Decay 0.25 (32), Filter 0.55 (70)	0.7
HH	Decay 0.08 (10), Tune 0.7 (89)	0.45
OH	Decay 0.34 (43), Tune shared with HH	0.4
CY	Decay 0.55 (70)	0.35
LTC, MTC, HTC	Tune 0.30, 0.42, 0.55 (38, 53, 70), Decay 0.32, 0.28, 0.24 (41, 36, 30)	0
CL, MA, CB	default	0
Lead, bass	default, no steps	1

Every knob not listed keeps its Knobs default, including Wave at CC 32 on BD1, BD2, and the toms. Master 0.7, 120 BPM, 16ths, INT.

Pattern "INIT", length 16, every track length 16, no step on. Tests that need steps arm their own. The factory bank (`engine/factory.h`, 21 kits with their patterns) follows INIT as programs 2 to 22; see `BUILD_GUIDE.md`.

CHOICE: the closed hat skips the open-hat steps. A closed trigger on the same sample chokes the open hat, so a closed hat on every step would leave the open hat silent.

Not in the engine, so not in the kit: a hat filter knob, a master Drive, and a global Accent amount. The sheet called the BD1 and SD sweep "Bend"; here that is the Pitch knob. Step bend stays off. One bar peaks at 0.943890 on the main.

Cleared voice

A voice that has never been triggered in the block under test outputs 0 on every sample. A drum step that is off does not trigger. A note-track rest does not open the gate. Neither event produces a new nonzero sample from a cleared voice.

Build guide (2026-10-06)

This tree builds the voice engine, the design-pack PDF, and a placeholder VST3. `make test` stays on the engine and the clock-switch law. It does not compile JUCE.

This tree

The voice engine is `engine/`. Panel art is unpainted. The editor in `plugin/` is a placeholder plate for listening in a host. JIDAI RACK is not changed by this repository. The equations in `SCHEMATICS.md` are the voice contract. `make test` builds `build/shogun_tests`, runs the checks in `TESTPLAN.md`, and runs the clock-switch law. That law test imports GraphForge from `~/graphforge/src`.

make strict compiles the engine, the tests, the web facade and the plugin sources with the warning flags JIDAI RACK uses (JUICE's recommended set plus -wfloat-equal -wimplicit-int-float-conversion -wshadow -wconversion -wdouble-promotion -werror) under clang++ and g++. It needs the JUICE checkout at JUCE_LINUX (default /workspace/JUCE) for the plugin headers.

Factory bank

Program 1 is INIT (the INIT kit and the empty pattern "001 INIT"). Programs 2 to 22 are the factory kits, each with its pattern "NNN name". The bank is data in one place:

File	Job
engine/factory.h	shogun::factory::kBank, kCount (21), kPrograms (22), programName(i), programJson(i), loadProgram(i, Patch&)
engine/factory_bank.inc	the 21 documents, generated; do not edit by hand
engine/patch.h	the saved-state document: Patch, parsePatch, applyPatch, capturePatch, writePatchJson (any sink, no libc; the wasm page saves with it) and patchToJson (native std::string)
scripts/make_factory.py	the kits and patterns as code; make factory runs it
tools/factory_fmt.cpp	writes the canonical text and measures the peaks (native renders at 44.1 and 48 kHz)

Each document is what the plugin saves inside <SHOGUN version=2>: sparse params by SECTION:LABEL id (missing ids keep INIT), mod rows, cables (none in the bank), cvAmt, inLaw and seq. The bank text equals patchToJson(parsePatch(text), false), so it is canonical and nothing in it goes through the alias table. make factory trims every kit until its 8-bar peak is -8 dBFS at the default master (within 0.35 dB), then writes the canonical text.

The engine loads a program with factory::loadProgram and applyPatch. The plugin lists the programs through getNumPrograms, getProgramName and setCurrentProgram (INIT, then the document through the same reader as a saved state; the parameters land at once, without the 5 ms glide), and the KIT and PATTERN displays open the same list. The web build exports sg_factory_count, sg_factory_name, sg_factory_load and sg_factory_kit; the page reads the bank back into its pattern and kit lists at load and plays a factory pattern from the document itself.

Embedding in JIDAI RACK

To run SHOGUN as a rack device, compile one engine translation unit:

- engine/shogun.cpp (everything else in engine/ and engine/voices/ is headers and .inc tables);
- include directories engine/ and third_party/jidai-common/include (the vendored shared headers, header-only);
- C++17, no other defines (SHOGUN_NO_FORMAT is only for the freestanding wasm build).

Entry points: shogun::Engine (prepare(fs, os), loadInit(), setParam/setParamNow, setHostTransport, setRunning, trigger/noteOn/noteOff, processSample(values, connected) with the kPorts jack buffer, mainL/mainR/aux, latencySamples), the parameter table in engine/params.h (kParams, findParam) and the jack table in engine/ports.h (kPortTable, findPort, resolvePort). The factory bank and the state reader are header-only: include engine/factory.h (it includes engine/patch.h and engine/factory_bank.inc) in the device source; there is no extra file to compile. Saved state is patchToJson / parsePatch.

The panel is data: plugin/layout/export_layout.py records the spec mockups and binds every control, and writes plugin/Source/PanelLayout.inc. scripts/check_panel_bindings.py (make panel-check, run by make test, make plugin and make probe-linux) fails when an interactive op on any tab has no bind, or a bind kind the editor does not handle. Regenerate the layout after changing the binders: python3 plugin/layout/export_layout.py.

If the rack reuses the JUICE processor and panel instead of its own wrapper, add plugin/Source/PluginProcessor.cpp and plugin/Source/PluginEditor.cpp (with plugin/Source/ on the include path for PluginProcessor.h, PluginEditor.h and PanelLayout.inc). plugin/Source/Probe.cpp is the test console only. make strict checks all of these with the rack's flags.

Rebuild the pack PDF from the markdown:

```
python3 scripts/build_design_pack_pdf.py
```

The script reads README.md, METHODOLOGY.md, SCHEMATICS.md, BUILD_GUIDE.md, TESTPLAN.md, and REPO_SETUP.md, in that order, and writes docs/SHOGUN_Design_Pack.pdf. It needs reportlab. It does not fetch audio code.

Engine

Piece	Job
Clock	INT and EXT, tempo, scale, the step counter that keeps running in EXT
Clock law	forge/graphs/switch_law.json. The switch chooses. Printed sample rows stay in the voice tests
Pattern	14 drum tracks, 2 note tracks, lengths, shuffle, shift, mute, per-step flags
Voices	One block per voice in SCHEMATICS.md, cleared to 0 until triggered
Outs	The pair map, the main sum, master after the sum
Tests	The names in TESTPLAN.md, against the printed 48 kHz rows

Host tempo wins while the host is playing. Otherwise the internal tempo, 60 to 180 BPM, default 120, is the clock. The display shows BPM and the step. EXT leaves both of those moving.

Trig jacks are rising edges through 1.0 V. In INT they are not read. In EXT they are the only triggers. Velocity on the edge is the accent in EXT. A gate with no velocity byte uses 127.

Live knobs apply in both modes. Pattern step on, flam, accent, and step bend apply only in INT.

Placeholder plate (2026-10-06)

make plugin configures plugin/ against the JUCE 8.0.4 tree already on this machine, then builds a universal (arm64 and x86_64) VST3 and a console probe. The deployment target is 11.0. Standalone is not a target on this machine: Command Line Tools do not include ibtool.

```
make plugin
make install-vst
```

make install-vst symlinks ~/Library/Audio/Plug-Ins/VST3/SHOGUN.vst3 to the Release artefact. FL Studio 2024 needs a plugin rescan after that link exists. The binary is an instrument, MIDI input is on, and the editor has a fixed 1200 by 672 logical size with a user scale.

The plate is flat greys and stock sliders. Each voice is one row: name, TRIG, level, and that voice's knobs. The name selects which voice the 16 step buttons edit. INT is the default and plays a beat: BD1 and bass on steps 1 and 9, snare on steps 5 and 13, hats on the eighths. EXT stays silent until TRIG or a MIDI note. MIDI notes 36 to 51 play BD1 through bass, in voice order. The clock parameter is the only writer of INT and EXT. TRIG and MIDI call the engine trigger path and leave the switch where it is.

The wrapper resamples the engine's 48 kHz stream to the host rate. Trig jacks stay unwired, so a note cannot act as a cable that forces EXT. Saved state is the parameter tree plus 16 step masks.

make plugin runs ShogunProbe, then ad-hoc signs the bundle so the module info stays inside the signature. The probe checks INT sound, EXT silence, MIDI and TRIG while EXT stays EXT, a bass note and its release, a 44.1 kHz host rate, a quiet tail after the steps are cleared, a restored step mask and clock, and that the plate paints. It writes /tmp/shogun-plate.png when given that path.

Panel

The jobs may match the instrument chapter: the same knob names, the same 14 drums, the two note tracks, the INT/EXT switch, and a Trig jack on each voice. The face must not be a trace of the hardware panel. No maker's logo or mascot from the source instrument. Draw a new plate. Level and master stay off the pattern, as in both manuals.

The INT/EXT switch is a SHOGUN control. Putting it on the plate does not make it a copy of a control on the source instrument, because that machine does not have this switch.

Outputs

Pairs, post level and post accent, before master:

- BD: left BD1, right BD2
- SD/RS: left snare, right rim
- HH/CY: left open plus closed, right cymbal
- CP: stereo clap
- TO/CO: LTC, MTC, and HTC at pans -0.7, 0, and 0.7
- CB/CL: left clave, right cowbell

Main is the sum of those contributions, plus maracas, lead, and bass at center pan, then the master. A patched pair does not remove its voice from the main.

Limits

The voice is the equation set in SCHEMATICS.md. A STAND-IN number stays marked until a measurement replaces it. A sampled kit is outside this design. A transistor netlist is outside this design: the manuals did not provide one. JIDAI RACK does not compile this repository, and this guide does not add it.

Test plan (spec v2.2, 2026-10-08)

The contract for the v2.2 engine (engine/), the plugin shell (plugin/) and the web build (web/). Each test below is a §15.4 test of SHOGUN_Redesign.md v2.2 (or a test this pass added), compiled into build/shogun_tests from tests/blocks.cpp, tests/voices.cpp, tests/engine.cpp, tests/mod.cpp, tests/factory.cpp. Every number is the line the test prints at 48 kHz, double precision; the tolerance column is what the named check uses. `make test` runs them (plus the forge law test), `make asan` runs them under ASan/UBSan.

Conventions: $u \in [0, 1]$ for every parameter (SECTION:LABEL ids); 1 V/oct with 0 V = C3 = note 48; accent volts 2.353/3.706/5.0 V; voice end at -90 dB (kQuiet 3.162e-5); OS 2× unless a row says otherwise (latency 23 base samples).

Result of the last run: **all 342 named checks passed** (`make test`, `make asan`; g++ 14.2 and clang++ 19.1 print identical output).

§15.4 tests not in this suite: `testRackLatencyComp` (belongs to jidai-rack, which this pass does not touch) and the v2.1 INIT-kit test (superseded by v2.2 §14.1: INIT ships an empty pattern; `testInitKitAndEmptyPattern` checks that instead).

DSP blocks (tests/blocks.cpp)

testDecayIsRateIndependent

§3.1 host-rate coefficients: the -60 dB time of one decay setting is the same at 44.1/48/96 kHz (closed form 940.802 ms). Tolerance: tol 0.1 ms.

```
-60 dB at 44100 Hz 940.816 ms (closed form 940.802)
-60 dB at 48000 Hz 940.812 ms (closed form 940.802)
-60 dB at 96000 Hz 940.802 ms (closed form 940.802)
```

testVoiceEndsAtMinus90

§3.6 a voice ends at -90 dB (kQuiet 3.162e-5): end sample = $\text{ceil}(\ln 6 \cdot \tau_a \cdot \text{fs} E) + \text{ceil}(\ln(1/k\text{Quiet}) \cdot \tau \cdot \text{fs} E)$, /M ceil. BD1 example 67,738, Decay 64 38,426. Tolerance: exact sample.

BD1 example end 67738, Decay 64 end 38426 (threshold 3.162e-05)
BD1 example ends at n = 67738 (spec 67,738); Decay 64 at 38426 (spec 38,426)

testSvfMinus3dB

§4.1 TPT SVF LP is -3.0103 dB at fc at every host rate. Tolerance: 1e-3 dB.

LP at fc 44100 Hz: -3.0103 dB
LP at fc 48000 Hz: -3.0103 dB
LP at fc 96000 Hz: -3.0103 dB

testResonatorTau

§4.2 resonator decay τ equals the target at every rate, also with the BD1 pitch sweep (quadrature damping correction).
Tolerance: 1e-3 ms.

tau at 44100 Hz 136.1950 ms (target 136.195)
tau at 48000 Hz 136.1950 ms (target 136.195)
tau at 96000 Hz 136.1950 ms (target 136.195)
tau with the BD1 pitch sweep 136.1958 ms

testResonatorPeak

§4.2 resonator impulse peak stays bounded across 35 Hz to 3 kHz. Tolerance: printed.

peak at 35 Hz 0.8700
peak at 140 Hz 0.9650
peak at 1000 Hz 0.9908
peak at 3000 Hz 0.9603

testRetriggerNoClick

§4.3 retrigger without a phase reset: max sample step 0.01005 (old reset 1.0); BD1 engine retrigger 0.0032 vs free-running 0.0013. Tolerance: limit 0.02 + free.

max sample step with a retrigger 0.01005 (old phase reset 1.0)
BD1 retrigger max step 0.0032 (free-running 0.0013), limit 0.02 + free

testLadderResponse

§4.4 ladder gain at fc for k 0/2/3.5 (exact -12.041/-6.021/+6.021 dB); at the verify FFT bin 1000.49 Hz -12.050/-6.038/+5.953 dB (spec -12.05/-6.04/5.95). Tolerance: 0.01 dB.

k 0.0 gain at fc -12.041 dB (exact -12.041), at the verify bin 1000.49 Hz -12.050 dB (spec -12.05)
k 2.0 gain at fc -6.021 dB (exact -6.021), at the verify bin 1000.49 Hz -6.038 dB (spec -6.04)
k 3.5 gain at fc 6.021 dB (exact 6.021), at the verify bin 1000.49 Hz 5.953 dB (spec 5.95)

testLadderSelfOsc

§4.4 no self-oscillation at k 3.8, self-oscillation at k 4.2; Newton ≤ 3 iterations. Tolerance: printed.

k 3.8 rms of the last 0.5 s 0, peak 0.0012
k 4.2 rms of the last 0.5 s 0.0759, peak 0.1074
Newton iterations max 3 mean 2.318, hard-drive peak 0.7465

testOtaSvfBounded

§4.5 OTA SVF at R 0 is bounded (impulse peak 0.1042 = verify). Tolerance: 1e-4.

R 0 impulse peak 0.1042 (verify 0.1042); R 0 hard input peak 6.4877

testHalfbandSpec

§3.4 decimators: stage 1 93 taps ≥ 110 dB stopband, stage 2 25 taps ≥ 120 dB. Stage 1 is the shared exact halfband `jidai::dsp::Halfband93` (jidai-common, polyphase, every even-offset tap 0 except the centre, as §3.4 asks). Stage 2 is SHOGUN's own 25-tap equiripple, which is not exact halfband; the shared header has no 4× stage. Tolerance: spec mins; 46 zero taps; 23 base samples per direction.

stage 1 (shared exact halfband, 93 taps, 46 even-offset zeros) stopband 111.64 dB ripple 4.53e-05 dB; stage 2 (25 t

testDecimatorDelay

§3.4 the decimator chain is a pure delay of 23 (2×) / 26 (4×) base samples. Tolerance: 1e-6.

2x output = input delayed 23 base samples, max error 1.22e-07
4x output = input delayed 26 base samples, max error 5.19e-07

testBlepAlias

§3.5 PolyBLEP saw alias at 2× (−60.1 dB at 1318.5 Hz). Tolerance: spec limits.

BLEP saw 1318.51 Hz at 2x, worst alias -60.1 dB
BLEP saw 3500.00 Hz at 2x, worst alias -52.3 dB

testTanhAaaaAlias

§3.5 ADAA tanh alias −77.8 dB (verify −77.8). Tolerance: 0.1 dB.

4987 Hz, drive 4, 2x ADAA worst alias -77.8 dB (verify -77.8)

testOuDriftStd

§5 OU drift: ρ 0.999556, std 3.990 cents over 20,000 s (target 4). Tolerance: 0.1 cent.

ρ 0.999556, std over 20,000 s 3.990 cents

testSmoothing

§3.2 5 ms one-pole smoothing coefficient at 48 kHz. Tolerance: 1e-6.

5 ms coefficient at 48 kHz 0.995842

testLevelLaw

§9.1 $g_{\text{Level}} = 1.4125 \cdot u^2$. Tolerance: 1e-4.

u 0.25 -> 0.0883
u 0.50 -> 0.3531
u 0.75 -> 0.7946
u 1.00 -> 1.4125

testCentrePan

§9.1 equal-power centre pan 0.707107. Tolerance: 1e-6.

centre 0.707107 / 0.707107

testGateHysteresis

JCS R3 Schmitt gate 1.0/0.5 V (the shared `jidai::jcs::Schmitt`, float volts); plain 0/5 V gates give one edge each. Tolerance: exact.

0.0 V -> edge 0 high 0
0.9 V -> edge 0 high 0
1.1 V -> edge 1 high 1
0.7 V -> edge 0 high 1
0.4 V -> edge 0 high 0
1.1 V -> edge 1 high 1

0/5 V logic gate, 4 gates -> 4 edges

testWaveBypass

§4.6 WAVE 0 with trims/SYM/SHAPE 0 (the default patch) is a true bypass (bit-identical). Any SYM at amount 0 is covered by testWaveAmt0AnySym. Tolerance: exact.

WAVE 0, trims/SYM/SHAPE 0 (default patch): bypassed 1, output bit-identical 1, level gain untouched 1.0

testWaveMigration

§4.6 v2.1 → v2.2 WAVE migration (macro + trim 2), max error 0. Tolerance: exact.

max error v 0..127 0; v32 m 0.126 trim2 -0.000; v90 m 0.354 trim2 -0.209; v127 m 0.500 trim2 -0.500

testWaveStageLaw

§4.6 stage law: s(0)=0; SYM +1 triangle and saw harmonic levels. Tolerance: 0.1 dB.

max |s(0)| 0; SYM +1 triangle: H1 -240.0 H3 -240.0 H4 -14.0 dB re 2F; saw: H2 -14.0 H3 -21.3 dB re 1F

testWaveAliasStatic

§4.6 2× ADAA alias levels (verify -60.0/-117.8/-121.9 dB). Tolerance: 0.1 dB.

398 Hz, WAVE 1.0, 2x ADAA alias -60.0 dB (verify -60.0)
220 Hz, WAVE 1.0, 2x ADAA alias -117.8 dB (verify -117.8)
1031 Hz, WAVE 0.5, 2x ADAA alias -121.9 dB (verify -121.9)

testWaveAudioRateVc

§4.6 audio-rate VC alias (verify -58.4/-113.7/-50.8 dB) and unsmoothed VC sidebands (+1.0 dB). Tolerance: 0.1 dB. VC → SYM moved from -115.3 to -113.7 dB with jidai-common 1.1.2: at WAVE 0.5 stage 3 sits at a = 0, so it is now a wire at any SYM; under 1.1.1 it ran ADAA on the identity, whose half-sample average also took 1.6 dB off the alias. The verify figure is verify_folder.py rerun (in scratch) with the 1.1.2 skip rule, a = 0 only; it gives -113.7 dB, the engine's number.

body 257.8 Hz, VC 1353.5 Hz, VC -> AMT depth 0.5: 2x ADAA alias -58.4 dB (verify -58.4)
body 257.8 Hz, VC 1353.5 Hz, VC -> SYM depth 0.5: 2x ADAA alias -113.7 dB (verify -113.7)
body 257.8 Hz, VC 1353.5 Hz, VC -> AMT + SYM depth 0.5: 2x ADAA alias -50.8 dB (verify -50.8)
VC sideband energy re harmonics 1.0 dB (unsmoothed reference +1.0, a 5 ms smoother gives -35.8)

testWavePlanBlock

jidai-common 1.1.2 per-block stage skipping (§4.6, TripleShaper::planBlock), the "amount 0, any SYM" rule: a stage is a wire for a block when its amount a is 0 for the whole block, whatever its SYM. With WAVE 0.25 (stage 1 live, stages 2 and 3 at a = 0) and SYM 2 0.6 and VC → SYM 3 0.5 set, a steady block makes stages 2 and 3 wires (SYM and VC → SYM never keep a stage). A moving block (ramp or modulation on an amount control) skips nothing. A live VC with AMT depth on stage 3 keeps it; the same depth with VC not live does not. A moving block keeps the ADAA half-sample delay on its a = 0 stages instead of switching to the wire (here with SYM 2 0.25 on the a = 0 stage; the steady twin makes that stage a wire). Tolerance: exact.

SYM 2 0.6, VC>SYM 3 0.5: wires (stage 1 2 3) steady 011, moving 000; VC>AMT on 3: live 010, not live 011; moving b

In the engine the block is one base sample (SHOGUN reads its parameters once per base sample). waveslot::control calls it steady when the 7 amount controls (WAVE, WAVE 1 to 3, VC→AMT 1 to 3) equal the previous base sample's and none is ramping (smoother) or modulated (matrix row, CV jack); SYM 1 to 3 and VC→SYM 1 to 3 may move. VC is live when the voice's VC LEVEL is above 0. Before 1.1.2 the plan read all 13 stage controls and a stage needed a = b = 0.

testWaveAmt0AnySym

§4.6 bypass, "amount 0, any SYM" (jidai-common 1.1.2 ShaperControls::isBypass ignores SYM, matrix SYM and VC → SYM). Block level: WAVE 0 with the trims at 0, five SYM 1 to 3 sets and five VC → SYM sets, SYM moving every 64 samples (steady and moving blocks), a live audio-rate VC, LEVEL COMP on and off, a VC → AMT depth with no live VC, at 48, 96 and 192 kHz (1×/2×/4×): every case is bypassed and every sample is bit-identical to the input and to the default bypass; LEVEL COMP is never touched (gain stays exactly 1). A live VC → AMT depth or a WAVE 3 trim of 0.01 is not a bypass. Engine level (tests/voices.cpp): BD1, BD2 and LTC at WAVE 0 with SYM 1 to 3 and VC → SYM 1 to 3 off centre, VC LEVEL 1 and a matrix row LFO 1 → SYM 2 render 9600 samples bit for bit like the default patch at 1×, 2× and 4×. Tolerance: exact.

```
WAVE 0, trims 0, SYM / VC>SYM / idle VC>AMT set: 60/60 cases bypassed, output bit-identical to input and to default
BD1/BD2/LTC at WAVE 0, SYM 1-3 / VC>SYM 1-3 off centre, live VC, LFO 1 -> SYM 2: 9/9 renders bit-identical to the d
```

Before 1.1.2 a SYM other than 0 at amount 0 ran ADAA on a straight line (a two-sample average: half a sample late, duller treble) and then LEVEL COMP. Now the stages are wires (their histories kept), the 8 Hz detector DC block is not fed and LevelComp::track() is not called, because §4.6 skips LEVEL COMP in bypass.

testWaveLevelComp

§4.6 LEVEL COMP keeps 200 random settings within -1.86..+0.20 dB (off: -13.9..+11.6). Tolerance: ±2 dB.

```
200 random settings, output re input RMS: comp OFF -13.9..11.6 dB, comp ON -1.86..0.20 dB
```

testWavePreVca

§4.6 PRE-VCA folder input follows the envelope (max deviation 0.0358 FS, verify 0.036). Tolerance: 0.002.

```
PRE-VCA folder-input peak vs envelope, max deviation 0.0358 FS (verify 0.036); at 250 ms 0.128
```

testShapeMorph

§4.6 SHAPE morph alias (-62.2/-109.1 dB = verify) and 0.000 % quadrature ripple. Tolerance: 0.1 dB.

```
398 Hz at 2x: SHAPE 1.0 (saw) alias -62.2 dB (verify -62.2), SHAPE 0.5 (tri) -109.1 dB (verify -109.1)
quadrature amplitude ripple at 35 Hz (2x of 48k): 0.000 %
quadrature amplitude ripple at 70 Hz (2x of 48k): 0.000 %
quadrature amplitude ripple at 400 Hz (2x of 48k): 0.000 %
quadrature amplitude ripple at 1000 Hz (2x of 48k): 0.000 %
```

Voices (tests/voices.cpp)

testKickBendDecays

§6.1 BD1 pitch sweep decays toward f_tune; bend -12 halves f(0). y(48)/y(480) locked at first build. Tolerance: 1e-5.

```
f(0) 83.814360 Hz, f(2400) 72.957296 Hz, f_tune 60.408707 Hz
y(48) 0.143497, y(480) -0.154601
bend -12 f(0) 41.907180 Hz
```

testVoiceEndsWhenQuiet

§3.6 every drum ends and the main output is exactly 0 afterwards; BD2 Decay 127 holds (env 0.70). Tolerance: exact.

```
BD1 example end 67738 (limit 3.162e-5), main exactly 0 after: 1, retrigger y(48) 0.143497 vs 0.143497
BD1 with Noise 127 ends at 67738
BD1 ends at n = 37751, then 0: 1
BD2 ends at n = 37768, then 0: 1
SD ends at n = 37768, then 0: 1
RS ends at n = 5974, then 0: 1
CP ends at n = 39880, then 0: 1
CL ends at n = 37751, then 0: 1
MA ends at n = 37768, then 0: 1
CB ends at n = 34470, then 0: 1
CH ends at n = 37768, then 0: 1
```



```
OH ends at n = 37768, then 0: 1
CY ends at n = 37768, then 0: 1
LTC ends at n = 37751, then 0: 1
MTC ends at n = 37751, then 0: 1
HTC ends at n = 37751, then 0: 1
BD2 Decay 127 active after 10 s: 1, env 0.700000
```

testDecayNoonIsShort

§3.6/§6.0 decay τ 8 / 75.902 / 720.137 ms at u 0/0.5/1; per-voice end samples (instant env 38,426, 0.2 ms VCA attack 38,443, CP one burst 38,971). Tolerance: exact sample.

```
decay_tau 8.000 / 75.902 / 720.137 ms at u = 0 / 0.5 / 1
Decay 64 decay-only end 38426 (spec 38,426); VCA voices (0.2 ms attack) 38443
BD2 ends at 38443 (want 38443)
CY ends at 38443 (want 38443)
OH ends at 38443 (want 38443)
CH ends at 38443 (want 38443)
CL ends at 38426 (want 38426)
MTC ends at 38426 (want 38426)
MA ends at 38443 (want 38443)
SD ends at 38443 (want 38443)
CP (one burst) tail opens at 528, ends at 38971 (want 38971)
CB at n = 38426: 0
```

testBd2CanHold

§6.2 BD2 Decay 127 holds at 0.70, 126 does not; click transient nonzero. Tolerance: 2e-4.

```
env(96000) Decay 127 0.702023, Decay 0 0.000000, Decay 126 0.056308
Tune 60 Tone 100 click at n = 10: 0.131217 (nonzero)
```

testBd2FullHolds

§6.2 BD2 full decay holds 0.70 for 10 s; next hit restarts at 1.0. Tolerance: 1e-6.

```
env at 10 s 0.700000 (S 0.70), next hit peak 1.000000
```

testTomFullEnds

§6.7 tom full decay ring ends at 215,434; a second hit restarts the ring. Tolerance: exact.

```
ring(192000) 0.550005 active 1, ring(194400) 0.202335, ends at 215434 (want 215434)
second hit at 2 s restarts the ring envelope: 0.999973
```

testSnareBendAtPitchZero

§6.3 SD tone 1 with bend at PITCH 0 and PITCH 127. Tolerance: 1e-5.

```
f1 466.028122 Hz at n = 0, 300.694079 at 3,840; bend 0 233.014061; Pitch 127 at 6,240 404.878529
```

testHatChoke

§6.6 CH chokes OH with a 1.5 ms fade (e^{-1} at +72 samples), OH ends +745. Tolerance: 1e-3.

```
OH env 0.986351 before choke, 0.972746 / 0.590000 / 0.357853 at +0 / +36 / +72 samples, OH ends +745
CH own tau 0.033008 s (not cleared by its own trigger)
```

testClapBurstCount

§6.5 clap burst starts 0/528/1056/1584; burst ratio $e^{\{528/144\}}$ exact. Tolerance: 1e-9.

```
count 4 starts: 0 528 1056 1584; count 1 starts: 1
burst1/burst0 at n = 1,589: 39.121283998 ( $e^{\{528/144\}}$  = 39.121283998)
```

testCpNoAlias

§6.5 burst centre below $0.45 \cdot f_s$ at 44.1 kHz. Tolerance: limit.

burst centre at $s = 15$: 9244.2 Hz; $0.45 \cdot f_s$ at 44.1 k = 19845.0 Hz

testNoonIsADrumMachine

§6.0 noon kit: BD1 f(100 ms) 89.931 Hz (spec says 70 ± 3 % but its own PITCH/ τ pins give 89.93: deviation), SD noise share $0.279 \geq 0.25$, tone peak 228.5 Hz (measured 100 to 400 ms: deviation). Tolerance: printed.

BD1 noon f(100 ms) 89.931 Hz, -60 dB time 524.3 ms, SOUND 0 WAVE 0.00 DRIVE 0.00
SD noon noise share (20-200 ms) 0.279, tone peak (100-400 ms) 228.5 Hz

testDistBypassAndDrive

§6.1 BD1 DRIVE 0 is a bypass, drive raises level (values locked at first build). Tolerance: $1e-5$.

n = 48: Dist 0 0.143497, Dist 1 0.143073, Dist 64 0.170431

testCenterPanIsNotHalf

§9.1 centre pan 0.707107 per side; CP Attack 0 tail only. Tolerance: $1e-6$.

MA centre: main L/voice 0.707107, R/voice 0.707107
CP Attack 0: silent before 528 1, L = R after 1

testIndividualOutStaysInMix

§9.2 patching OUT does not remove the voice from the mix; master -6 dB halves main only. Tolerance: $1e-6$.

n = 48: OUT 0.712506 V, main L 0.100764 (patched), main L 0.100764 (unpatched), master -6 dB main 0.050382 OUT 0.712506 V

testRetNormal

§9.2 RET normal: unpatched RET = OUT; RET patched to 0 V silences the voice in the mix. Tolerance: exact.

RET unpatched main peak 0.124851; RET patched 0 V main peak 0, OUT peak 0.8828 V

testVelNormal

§4.8 VEL normal = accent volts 2.353/3.706/5.0 $\rightarrow$ g_vel 0.55/0.78/1.00. Tolerance: $1e-4$.

accent 1 $\rightarrow$ VEL normal 2.353 V, g_vel 0.5500
accent 2 $\rightarrow$ VEL normal 3.706 V, g_vel 0.7800
accent 3 $\rightarrow$ VEL normal 5.000 V, g_vel 1.0000

Engine, clock, ports (tests/engine.cpp)

testExtBypassIgnoresPattern

§12.2 forge pin: EXT ignores the pattern, TRIG plays (g_vel 0.819291, $y(10) = \text{ref} \times g$); back in INT the pattern plays. Tolerance: $1e-6/1e-9$.

BD1 silent over step 0 in EXT: 1; display 1 $\rightarrow$ 2; TRIG vel 100: g_vel 0.819291, $y(10)$ 0.031264 = 0.038160×0.819291
after INT: 2 pattern hits (flam pairs) in the next bar, first at +12988; TRIG edge ignored

testIntIgnoresTrigJacks

§12.2 forge pin: INT ignores TRIG jacks. Tolerance: $1e-9$.

TRIG edge in step 1 (INT) silent: 1; step 0 $y(10)$ 0.038160 = $\text{ref } 0.038160 \times 1.000000$

testRestIsSilent

§10.2 rest steps output 0 (BD1/LEAD/BASS). Tolerance: exact.

BD1 rest step outputs 0: 1
LEAD rest step outputs 0: 1
BASS rest step outputs 0: 1

testSoloMutesOtherVoices

§9.1 SOLO isolates a voice without moving the counter; MUTE wins. Tolerance: exact.

soloed SD main vs SD alone max diff 0; counter same 1; solo off main vs no-solo diff 0
soloed + muted SD direct trigger main peak 0

testShuffleSurvivesOddLength

§10.2 swing on odd-length tracks (hits 0 8000 12000 20000 24000 32000). Tolerance: exact.

delay 2000 samples; hits: 0 8000 12000 20000 24000 32000

testInitKitAndEmptyPattern

§14.1 INIT kit + empty 16-step pattern "001 INIT" (replaces the v2.1 INIT-kit test). Test beat peak -16.05 dBFS. Tolerance: 1e-4.

pattern '001 INIT', every track 16 long: 1, empty: 1, params at INIT: 1, one bar silent: 1
test beat main peak 0.157557 (-16.05 dBFS), OH on step 3 env max 1.000

testControlOutsZeroLatency

§3.4/JCS: ENV and gate outputs are at n with no OS latency; audio OUT lags 23. Tolerance: exact.

step at n = 0: 2x BD1:ENV 3.8989 V, MOD:LD GATE 5.0 V at n = 0; BD1:OUT lag 23 samples

testOsLatency

§3.4 latency 0/23/26; every audio path (main, OUT, AUX) lags exactly that. Tolerance: ±1 (ADAA half sample).

latencySamples 1x/2x/4x = 0 / 23 / 26
2x lags main 23, BD1:OUT 23, AUX 1 23; voice + bus + master drive main 22
4x lags main 26, BD1:OUT 26, AUX 1 26; voice + bus + master drive main 25

testPitchVoct

§7.1 1 V/oct, 0 V = C3 (note 48): NOTE OUT and NOTE IN exact, V/OCT +1 V = 12 st. Tolerance: 1e-6.

NOTE OUT note 36 = -1.0000 V, flag 0
NOTE OUT note 48 = +0.0000 V, flag 0
NOTE OUT note 60 = +1.0000 V, flag 0
NOTE OUT note 72 = +2.0000 V, flag 0
NOTE OUT note 24 = -2.0000 V, flag 0
NOTE OUT note 108 = +5.0000 V, flag 0
NOTE IN -1 V -> note 36.0000, pitch error 0.00000 cents
NOTE IN +0 V -> note 48.0000, pitch error 0.00000 cents
V/OCT +1 V adds 12.000000 semitones
NOTE IN +1 V -> note 60.0000, pitch error 0.00000 cents
NOTE IN +2 V -> note 72.0000, pitch error 0.00000 cents

testPitchRail

§7.1 NOTE OUT rail ±5 V with over-range flag. Tolerance: exact.

note 109 (OCT +1 on 97) +5.0000 V flag 1; note -13 -5.0000 V flag 1; max |NOTE OUT| over -40..150 5.0000

testTuningIsNotTheVoltLaw

§7.1 TUNE/A4 move the sound, not NOTE OUT (+1.0000 V, 278.4426 Hz). Tolerance: 1e-4.

TUNE +100 c, A4 442: NOTE OUT +1.0000 V, audible f 278.4426 Hz (law 278.4426, untuned 261.6256)

testLin55Migration

§12.3 BASS:HZ/V → BASS:NOTE with $V' = \log_2(\max(V, 1e-3)) - 1.25$ (2.0 V → 110.000000 Hz; 0 V and below read the 1 mV floor, -11.215784 V, since jidai-common 1.1.2); TOM:PITCH fans out to 3 toms at CV AMT 1/12. Tolerance: 1e-6 (floor 1e-12, exact for $V \leq 1$ mV).

```
BASS:HZ/V -> BASS:NOTE, law 1
2.0 V plays 110.000000 Hz (0.000000 cents from 110)
lin55 0 V -> -11.215784 V, -2 V -> -11.215784 V, 1 mV -> -11.215784 V (floor log2(1e-3) - 1.25)
```

testJackIdsAndTypes

§13.1/13.2: 151 ports, ids exact, unique, types/roles, plainVoltGates. §13.2 listed 153; CLOCK:FILL IN (never read) and MOD:LANE A (a constant 0 V) were removed so the table is exactly the panel bay. Tolerance: exact.

```
151 ports, ids exactly as 13.2 (less FILL IN / LANE A): 1, unique 1, type/role errors 0, plainVoltGates 1
```

testBayMatchesPortTable

The declared port table is the panel bay: every port in kPortTable has exactly one jack: bind among the JACK rows of plugin/Source/PanelLayout.inc (compiled into the test with the editor's row shape), and every bay jack names a port in kPortTable. Tolerance: exact.

```
bay 151 jacks, ports 151, missing 0, unknown 0, dups 0
```

testRemovedPortCablesDropped

Old documents with cables on the removed CLOCK:FILL IN and MOD:LANE A still load. parsePatch drops each cable whose end does not resolve and reports it (Patch::droppedCables, droppedRemoved for removed ports, droppedIds with the first four as text); the good cable (MOD:LFO 1 → BD1:PITCH) is kept, applyPatch runs and BD1 plays. Checked in the bare, SHOGUN/ and SHOGUN#1/ forms; isRemovedPort knows both ids in any SHOGUN form and refuses another device's prefix. The plugin reports the same through droppedCables() / loadReport() (written to the JUCE log; probe check "removed jacks") and the web page through sg_patch_dropped() (console warning; web/test_wasm.mjs law "old FILL IN / LANE A cables"). Tolerance: exact.

```
3 forms (bare, SHOGUN/, SHOGUN#1/): 3/3 load, drop FILL IN + LANE A, keep LFO 1 -> BD1:PITCH, play
```

testJackIdsJcsShared

JCS R6/R14 through the shared jidai-common headers. Every one of the 151 port ids and every alias-table id is parsed whole by jidai::jcs::parseJackId in three forms: bare, SHOGUN/ and SHOGUN#1/. Labels keep '/', spaces and digits:

LEAD:V/OCT, LEAD:HZ/V OUT, MOD:LD GATE, MOD:LFO 1. resolvePort maps each form (and SHOGUN#12/) to the same port. Another device's prefix is refused. The three v2.0/2.1 names with no SECTION: (MIX L, MIX R, LFO OUT) are not R6 ids; they are matched whole against the alias table. All 454 parameter ids are R6 SECTION:LABEL ids. Role colours come from the shared table. Tolerance: exact.

```
ports 151/151 round trip whole in 3 forms (labels with '/' 2, with spaces 20, ids with digits 24)
alias-table ids 25, R6 ids round trip 22, legacy non-R6 names matched whole 3: [MIX L] [MIX R] [LFO OUT]; foreign p
shared AliasTable 6 renames (BASS:HZ/V -> BASS:NOTE, Lin55ToVoct(2.0 V) = -0.25 V); shim: fan-out 2 (shared add of
```

Since jidai-common 1.1.1 (law floored at 1 mV since 1.1.2) the one-to-one renames (LEAD/BASS:HZ/V → :NOTE with AliasLaw::Lin55ToVoct, LEAD/BASS:HZ/V OUT → :NOTE OUT, SD:SNAPPY → SD:TONE, LFO:OUT → MOD:LFO 1) are rows of the shared jidai::jcs::AliasTable, resolved with its resolve(); the engine applies the returned AliasConversion. The test also shows why the rest stays in a SHOGUN shim: the shared table refuses a second target for one old id (HAT:DECAY, TOM:PITCH fan out) and refuses ids with no SECTION: (MIX L, MIX R, LFO OUT).

Before this change, no bare id was mis-split. SECTION and LABEL never contain ':', so a first ':' split was right, and the engine matched ids whole. What failed was the prefixed forms:

- The plugin state loader stripped only SHOGUN/, so every SHOGUN#N/ id failed: the 153 ports of the time plus 11 alias names.

- The web `sg_port_find` matched only bare ids, so all 153 (then) failed in both `SHOGUN/` and `SHOGUN#N/` forms.

Now every external id goes through `resolvePort / portFromId` (`engine/ports.h`, which calls `parseJackId`): the plugin state loader (cables, CV AMT, inLaw), editor jack binds, wasm `sg_port_find`, and the web compat patch loader. The engine also builds in the freestanding wasm target, with small `<string>`, `<string_view>` and `<optional>` shims in `web/wasm/include`. The editor's mod-destination menu splits parameter ids with `parseJackId`.

testHostLock

§10.1 host lock: ppq 7.25 → step 29; a transport jump re-syncs at block sample 0. Tolerance: exact.

ppq 7.25 at 1/16 -> step 29; before jump step 29; after jump to 0 synced at block sample 0, BD1 hit yes

testAccOutFollowsPattern

§13.2 ACC OUT = step accent volts while the step is on, in INT and EXT (EXT voice silent). Added this pass. Tolerance: 1e-6.

INT: ACC OUT step 0 5.000 V, step 4 2.353 V, step 8 (off) 0.000 V; BD2 played 1
EXT: ACC OUT step 0 5.000 V, step 4 2.353 V, step 8 (off) 0.000 V; BD2 played 0

Modulation (tests/mod.cpp)

testModSumLaw

§8.4 $u_{\text{eff}} = \text{clamp}(u_{\text{base}} + \Sigma d \cdot C(s) \cdot \text{via} + \text{AMT} \cdot V/5)$; FREE rate law; engine matrix path matches. Tolerance: 1e-4.

law: $0.5 + 0.18 = 0.68$ (tau 170.62 ms); + 2.5 V -> 1.00; $0.2 - 0.3 \times \text{VEL } 0.5 = 0.05$; SOUND $0.40 + 0.10 \rightarrow 8$
FREE rate 0.0100 / 0.6325 / 40.0000 Hz at $u = 0 / 0.5 / 1$
engine: DECAY $u_{\text{eff}} 0.6800$ (tau 170.62 ms), +2.5 V -> 1.0000; TUNE via VEL 0.5 -> 0.0500; SOUND index 8

testOwnVoiceRetrigIndependent

§8.3 OWN VOICE LFO instances restart per voice independently. Tolerance: 1e-6.

$n = 12,000$: BD1 0.000000 SD 0.000000; $n = 18,000$: BD1 -1.000000 SD 1.000000; BD1 retrigger: BD1 0.000000, SD 1.000000

testLfoHostPhase

§8.3 SYNC FREE-RUN re-anchored $\phi = \text{frac}(\text{ppq}/D)$: 4.24e-13 cycles over 600 s (verify 6e-13). Tolerance: 1e-9.

worst re-anchored phase error over 600 s: 4.24e-13 cycles (limit 1e-9)

testLfoDeclick

§8.2 declick $\tau = \max(0.25 \text{ ms}, \text{SLEW} \cdot T/4)$: max step 0.1460, HF -35.77 dB (verify 0.146/-35.8). Tolerance: limits.

40 Hz square, PolyBLEP + 0.25 ms declick: max step 0.1460 (limit 0.15), energy above 2 kHz -35.77 dB (limit -35)

testShSlew

§8.2 S&H SLEW 1 at 1/4 120 BPM: 63 % at 125.021 ms (τ 125 ms). Tolerance: 0.1 ms.

SLEW 1 at 1/4, 120 BPM: tau 125 ms, 63 % at 125.021 ms

Factory bank (tests/factory.cpp)

The bank in `engine/factory.h` (INIT, then 21 kits with their patterns; `BUILD_GUIDE.md` "Factory bank"). 21 named checks.

testFactoryPresetsLoad

Every program parses with the shared reader (engine/patch.h); 16 to 24 kits after INIT; each pattern is named "NNN" + the program name; the engine holds exactly the loaded patch (applyPatch then capturePatch: every parameter as a host float, every mod row and step); no content that needs the alias table (no cables, no input laws, CV AMT 1); the Acid kit has at least 8 BASS steps; program 0 is INIT, identical to Engine::loadInit(). Tolerance: exact.

22 programs (INIT + 21 kits) load: 529 parameters off INIT, 611 steps (12 ratchets, 4 flams, 35 probability, 22 mics)

testFactoryRoundTrip

The bank text is the canonical writer's output (patchToJson(parsePatch(text), false)). Each program is loaded into an engine, saved in full (the plugin's state), reloaded into a new engine and saved again: the two saves are byte-identical and every parameter, mod row, cable, CV AMT, input law and step is equal. Tolerance: exact.

22 programs saved (317396 bytes of state), reloaded and saved again

testPatchWriterNoLibc

The libc-free writer (writePatchJson on patchjson::BufOut, which the wasm page saves with) against the libc one: fmtG gives the bytes of snprintf("%. *g") at every precision 1 to 17, shortestNum and shortestFloat give the text putNum and putFloat choose with snprintf and strtod (doubles from random bits, [0, 1), k/127, every power of two from 2⁻¹⁰⁷⁴ to 2¹⁰²³, subnormals, signed zero; floats from random bits), and every program's document is byte-identical to patchToJson, sparse and full. Tolerance: exact.

442510 %. *g texts, 182206 doubles and 182206+ floats, 22 programs x 2 documents

testFactoryRenderLevels

Each kit with its pattern, from a fresh engine at the default master, renders 2 bars at 44.1 and 48 kHz: every peak is above -40 dBFS (not silent) and at or below -6 dBFS. Tolerance: the limits.

21 kits x 44.1/48 kHz, 2 bars from a fresh engine, default master: peaks -10.47 (Jungle Break Roller) to -8.00 dBFS

testFactoryNames

Names are unique (case-insensitive), fit the pattern name, and carry none of 50 banned brand, gear, model-number, artist and trademarked genre words. Tolerance: exact.

22 names, 50 banned words checked

Plugin shell (ShogunProbe, make probe-linux)

The probe builds the real ShogunAudioProcessor/editor (JUCE 8.0.4) and checks the shell laws of §15.0 step 4. The three program checks are the factory bank through the host program list: the count and names, each kit playing 2 bars between -40 and -6 dBFS at 44.1 and 48 kHz (the same peaks as testFactoryRenderLevels), and each loaded program's state reloading to the same bytes. The panel checks press the real keys through the editor's binds (pressBind): the KIT and PATTERN arrows step every program with wraparound on all 8 tabs, SRC cycles CLOCK:SOURCE and the engine follows the host tempo only on HOST, A/B recalls two full snapshots exactly and copies either onto the other, undo/redo restores kit loads, knob gestures and step edits (a no-op click keeps redo; 64 levels), the MOD  keys arm a source and a knob click adds the row, UI SCALE resizes, the OS keys and offline 4x set their parameters, RE-ROLL UNIT changes the unit serial in the engine and the saved state, and VELOCITY CURVE shapes MIDI velocity. 26 checks, last run all PASS:

```
PASS params count 454 (table 454), float 0..1 SECTION:LABEL ids, mismatches 0
PASS latency 0/23/26 samples
PASS host rate engine fs 44100.0 Hz at host 44100
PASS INIT silent peak 0.000000, pattern '001 INIT'
PASS MIDI 36 -> BD1 peak 0.149777
PASS INT test beat peak 0.116818 = -18.65 dBFS
PASS EXT ignores pattern peak 0.000000
PASS host lock ppq 7.25 -> global step 29 (bar-of-16 display 14), step-29 hit peak 0.116818
PASS aux 1/2 bus channels 4, aux peak 0.149767, main peak 0.000000
```

```

PASS programs 22 (INIT + 21 factory kits), names 22
PASS programs play 21 kits x 2 rates, 2 bars, peaks -10.47 .. -8.00 dBFS
PASS program state 21/21 programs save and reload to the same state
PASS state round trip XML <SHOGUN version=2> JSON 11564 chars; params/pattern/lock/mod/cable restored
PASS alias load cables 4 (1 + 3 toms) = 4, BASS:NOTE law 1, MTC:PITCH CV AMT 0.083333
PASS old HZ/V plays BASS:HZ/V cable at 2.0 V -> 110.000000 Hz (0.00000 cents from 110), law 1
PASS removed jacks old FILL IN / LANE A cables dropped 2 (removed 2), kept 1; report: SHOGUN: dropped 2 saved cables
PASS editor 1200x672, bay jacks 151/151, ops 2550
PASS tab renders 8 PNGs in /workspace/shogun/build/plugin-linux/tabs
PASS KIT/PATTERN arrows 352 steps over 22 programs on 8 tabs, both arrow pairs wrap INIT <-> Lo-Fi Tape Wobble,
PASS SRC key SRC INT > SRC EXT > SRC HOST > SRC INT, right-click SRC EXT > SRC INT; engine tempo 120.0 / 120.0 / 1
PASS A/B compare B = copy on first visit, A/B recall exact (BD1:DECAY 0.774 / 0.900, SD step 4), copy A>B and B>A
PASS undo/redon kit load + knob gesture + step edit undone and redone exactly; no-op click keeps redo; 64 levels at
PASS ASSIGN keys LFO 1 > BD1:DECAY +50 % in slot 1, AT > BD2:TUNE in slot 2, second press cancels
PASS UI scale keys 150% = 1800 px, 75% = 900x504, 100% = 1200
PASS OS keys badge 2x > 4x, right-click back; offline 4x sets OFFLINE (OS stays 2x)
PASS RE-ROLL UNIT 0x5A31C0DE > 0x74C37F51, engine + saved state follow (SN 0x74C3-7F51)
PASS VELOCITY CURVE velocity 40 peaks HARD 0.035 < LINEAR 0.063 < SOFT 0.094 <= FIXED 0.150
ShogunProbe: all checks passed

```

Panel bindings (make panel-check, scripts/check_panel_bindings.py). Reads plugin/Source/PanelLayout.inc and the editor's bind table: every interactive op (KEY, KNOB, LCD, toggle, jack, grid cell) on every tab must carry a bind, and every bind prefix must be one the editor handles. Runs in make test, make plugin and make probe-linux.

```
check_panel_bindings: 860 interactive ops, 0 dead
```

Tab renders: build/plugin-linux/tabs/tab_<i>_<name>.png (1200 × 672) next to the spec mockups; mean absolute pixel difference per tab (0 to 255): MAIN 4.7, VOICE 5.8, GRID 12.4, MOD 7.5, ROUTE 8.6, FX/MIX 6.0, SEQ/MIDI 5.0, GLOBAL 4.3. The differences are the mockups' illustrative data (fake pattern, matrix rows, cables, meter levels), which the plugin replaces with live state (INIT: empty), plus font rasterisation, and the keys this pass removed or rebound (FILL, ROLL, MUTE GRP, SCENE, LEARN, LOCK RND, lane editor, NOISE FLOOR, TRANSPOSE, SAVE AS DEFAULT).

Web build (make web)

build/shogun.wasm is the same engine (freestanding, -DSHOGUN_NO_FORMAT), run at the AudioContext rate. web/test_wasm.mjs runs web/parity_scenario.txt through wasm and the native build of the same facade (build/web_parity) and compares every sample, then checks the bay and LFO laws, the jack-id forms and the factory bank. The scenario ends by loading every factory program (and one rotated for a chain, and INIT): after each load the wasm state hash (parameters as floats, mod rows, pattern) must equal the native one exactly, which checks the freestanding number reader against strtod. After the switch to the shared exact-halfband stage 1, the largest wasm/native difference went from 9.68e-10 to 3.74e-9, and with jidai-common 1.1.1 (exact C3 in the synth pitch) to 3.77e-9 (float32 output, same code on both sides; the wasm build imports JS Math):

```

200608 samples, peak 0.341, largest wasm/native difference 5.00e-10 at 188022; 23/23 loaded-state hashes equal (fa
wasm matches the native engine
ok  CLK OUT into BD1 Trig, INT: silent
ok  CLK OUT into BD1 Trig, EXT: fires
ok  ACC OUT into BD1 Trig, EXT, no loud step: silent
ok  ACC OUT into BD1 Trig, EXT, one loud step: fires
ok  ACC OUT into BD1 Trig, INT, one loud step: silent
ok  LFO 1/16 at 120 BPM is 8 Hz (period 6000.00 samples)
ok  LFO sine at amount 1 spans 0 to 5 V around 2.5 V (0.0002 .. 4.9998)
ok  LFO starts at phase 0 on transport start (2.5000 V)
ok  LFO amount 0 is 0 V
ok  LFO sample and hold moves only in the 5 ms after each cycle start (0 stray changes)
ok  44.1 kHz: rate 44100, latency 23, params 454, ports 151, INIT peak 0
ok  44.1 kHz: BD1 hit peak 0.1498
ok  jack ids: 151/151 resolve in bare, SHOGUN/ and SHOGUN#1/ forms (e.g. LEAD:V/OCT 115); legacy names ok; foreign
ok  LFO OUT into BD1 PITCH changes the kick
ok  factory bank: 22 programs (INIT + 21), unique names 22, 42 renders peak -10.47 .. -8.00 dBFS
ok  documents: 22/22 programs write the native bytes (317396 bytes), 22 reload to the same text and state
ok  old FILL IN / LANE A cables: load 1, dropped 2 (on removed jacks 2), LFO 1 -> BD1:PITCH kept true

```

The page setters now leave a value alone when it equals what the panel reads back (a knob's CC, a level, a track's shuffle and shift, a step's bend, the clock steps), so re-sending a panel keeps the engine's finer value; the scenario's largest difference moved from 3.77e-9 to 5.00e-10 with that (still float32 output of the same code on both sides). The new documents law checks the wasm sg_state_json against the native writer for every program.

Page save round trip (web/test_page.mjs, **headless Chrome**). web/shogun.html with a test appended, in a real browser. For every program (INIT and the 21 kits): load it, edit one visible control (a knob, a step on/off, an accent, TEMPO, a track LENGTH, a LEVEL, LFO AMOUNT or LFO DIVISION), SAVE, load the saved pattern again. The saved document must differ from the factory document only in the edited field; the reloaded panel must show the edit; the saved text must be a fixed point of load and save; the engine the page plays from (before and after the reload) must hold the saved document, apart from what the page owns (master, solo, INT/EXT, cables); a second SAVE without edits must give the same text. The LFO knobs must be the kit's LFO 1 and the LFO tab must list all four LFOs.

```
test_page: 22 programs: 22 saves change only the edited field, 22 reload showing the edit, 22 saved texts are fixed
test_page: the web save round trip keeps every field
```

jidai-common (vendored shared headers)

third_party/jidai-common is jidai-common 1.1.2, jidai-collection redesign/jidai at 8a4b5ae (8a4b5aefc3d6c2ac8900cc2e88db8feecf8440a8), copied verbatim with git archive (see VENDORED.md; the previous copies were 1.1.1 at 24ee621 and 1.1.0 at 9d6e382). SHOGUN uses these parts of it:

- jcs::Schmitt for every trigger, clock, reset, run and gate input;
- jcs::pitch kc3Hz (exact 130.8127826502993 Hz; the synth voice pitch), note, voltsForNote, clampPitch, lin55ToVoct (through the alias law);
- jcs::Role, roleInfo and roleArgb;
- jcs::parseJackId, isValidLocalId and AliasTable with AliasLaw / AliasConversion;
- dsp::TripleShaper (with planBlock), ShaperControls (with isBypass), ShaperStage, macroAmounts, LevelComp and DcBlocker;
- dsp::Halfband93, Downsampler2x and Upsampler2x (decimator and RET upsampler stage 1).

Removed local copies: engine/jidai_local.h and engine/jidai/dsp/TripleShaper.h (at 9d6e382); with 1.1.1 also the per-stage wire workaround in engine/wave_shaper.h, the local lin55 formula in engine/shogun.cpp and the local alias table in engine/ports.h.

Wrapper: engine/wave_shaper.h, class shogun::WaveShaper with shogun::TripleShaperParams: one shared TripleShaper, planned once per block by setParams(p, steady, vcLive), plus SHAPE morph, PRE-VCA and the v2.1 migration.

Workarounds removed with 1.1.1 (upstream now covers them):

1. **Wire rule.** Was: three shared TripleShaper instances (one live stage each) plus an ADAA-at-zero replacement, to keep the spec's whole-render rule against the per-sample skip. Now: the shared planBlock(ct1, steady, vcLive) once per block (testWavePlanBlock).
2. **lin55.** Was: a local $\log_2(\max(V, 1e-3)) - 1.25$ because the shared function was 1.9e-7 V low. Now: the shared lin55ToVoct through AliasLaw::Lin55ToVoct. 2.0 V still plays 110.000000 Hz (0.00000 cents). With 1.1.2 the shared law is $\log_2(\max(V, 1e-3)) - 1.25$, spec §12.3 word for word: $V \leq 1$ mV (0 V and negative volts included) gives -11.215784 V, where 1.1.1 gave -5 V (the rail) for $V \leq 0$ and went unbounded below for $0 < V < 1$ mV. The 1.1.1 difference listed here is gone (testLin55Migration checks the floor). Both values are below the lowest playable note.
3. **Alias table.** Was: SHOGUN's own table with fan-out and law codes. Now: the shared AliasTable with input laws for every one-to-one rename.

Still on SHOGUN's side, and why:

1. **Alias shim** (engine/ports.h, kPortFanOuts, kPortLegacyNames). The shared AliasTable maps one old id to one canonical id; add() refuses a second target for the same old id. HAT:DECAY → CH + OH:DECAY and TOM:PITCH → LTC/MTC/HTC:PITCH are one-to-many, and TOM:PITCH also sets each new cable's CV AMT to 1/12 (a cable amount the user can edit, not a hidden volts conversion). MIX L, MIX R and LFO OUT have no SECTION:, so isValidLocalId() is false and add() refuses them. testJackIdsJcsShared shows both refusals.

2. **LEVEL COMP detector floor.** The shared LevelComp floors both detector powers at 1e-30 (now documented upstream). Adopted as is; see the bit-identity numbers below.

3. **Detector input type.** jcs::Schmitt takes float volts; SHOGUN casts its double jack volts. Only matters within 6e-8 V of the 1.0 V and 0.5 V thresholds.

4. **Halfband stage 2.** The shared header has no 4× stage, so stage 2 (25 taps) stays SHOGUN's. Latency 0/23/26.

5. **Umbrella header.** The engine includes single headers; the freestanding wasm build has small <string>, <string_view>, <optional> and <vector> shims for JackId.h (load time only). The plugin and tests include jidai/CableStandard.h.

jidai-common 1.1.2 (output changes). (1) A WAVE stage at amount 0 is a wire at any SYM, and WAVE 0 with any SYM is the full bypass: stages, detector DC block and LEVEL COMP skipped (engine/wave_shaper.h); WaveSlot::control calls a block steady from the amount controls only (engine/voices/common.h). (2) HZ/V → NOTE is floored at 1 mV. Tests changed: testWavePlanBlock (to the new law; SYM on a = 0 stages stays a wire), testWaveAudioRateVc (VC → SYM -115.3 → -113.7 dB, stage 3 now a wire), testWaveBypass (label only), testLin55Migration (floor check added), and the new testWaveAmt0AnySym (block and engine). Every other printed number in the suite is unchanged (the full log against 1d7826f differs only in those lines). SHOGUN does not call LevelComp::track() in bypass or report groupDelay() to the host.

Golden render with 1.1.2. The scratch busy-patch render described under "Strict build" below (SHA-256 of the raw doubles, first 16 hex digits), against 1d7826f (1.1.1):

Build	1.1.1 (1d7826f)	1.1.2
g++ 14.2 and clang++ 19.1, -ffp-contract=off	4feda137ff4c35a9	4feda137ff4c35a9
g++ -mfma -ffp-contract=fast	01f7a8f927921fbf	01f7a8f927921fbf
clang++ -mfma -ffp-contract=on	4fc3e37230bd0a01	4fc3e37230bd0a01
clang++ -mfma -ffp-contract=fast	126f554cd6e62129	126f554cd6e62129

It does not change: that patch has no WAVE stage at amount 0 with a SYM other than 0 (BD1 SYM 2 is on a live stage, the a = 0 stages have SYM 0) and no HZ/V cable, so neither 1.1.2 output change (the WAVE amount-0 wire, the lin55 1 mV floor) reaches it. The same patch with SYM set on amount-0 stages (BD2 SYM 3 0.9, LTC SYM 3 0.2, MTC SYM 1 0.85 and VC → SYM 2 0.1 at WAVE 0, HTC SYM 2 0.7 at WAVE 0) does change, because of the WAVE amount-0 wire: 1.1.1 e995ecda8606aaa7 (g++ and clang++, contraction off; g++ FMA fast c96ac2668021ac76, clang++ FMA fast 28dfe551bf491b2d) becomes 4feda137ff4c35a9 (FMA fast 01f7a8f927921fbf / 126f554cd6e62129), the busy patch's own hashes: under 1.1.2 those SYM settings have no effect at all, as the rule says. The lin55 floor is below every playable note and is checked by testLin55Migration, not the render. The test log is byte-identical across g++ and clang++ with contraction off, g++ FMA fast and clang++ FMA on; clang++ FMA fast differs in one line (testWaveMigration max error 2.22e-16 instead of 0, the same as under 1.1.1), all 339 checks passing in every build.

Bit-identity of the wave tests. A scratch dump program renders every sample the eight wave tests compute.

- 1.1.1 vs the 9d6e382 build (54ff487): all eight are byte-identical, with g++ and with clang++. The per-block plan gives the same wire decisions as the old static rule for these renders, and the shared AdaaStage computes the same ADAA values on (0, 0) samples.

- 9d6e382 vs the local shaper (dce82f8): identical except testWaveLevelComp with LEVEL COMP ON (max abs difference 9.86e-7, relative 5.4e-7, from the 1e-30 floor; under 1.5e-7 after 10 ms, 0 after 150 ms). Every printed wave number is unchanged.

Exact C3. The synth voice pitch is now $\text{kc3Hz} \cdot (\text{A4}/440) \cdot 2^{((\text{note} - 48)/12 + \text{cents}/1200 + \text{V/OCT})}$ instead of $\text{A4} \cdot 2^{((\text{note} - 69)/12 + \dots)}$. The two are equal in exact arithmetic; doubles differ in the last bits. No printed test number changed. The tests' own reference pitches stay A4-based ($440 \cdot 2^{((n - 69)/12)}$), an independent check of the C3 constant. No test pinned 130.8128.

Strict build (make strict, JIDAI RACK flags). The engine, the tests, the web facade, tools/measure_calib.cpp and every plugin/Source file compile with JUCE's recommended warning flags (juce_recommended_warning_flags) plus -Wfloat-equal -Wimplicit-int-float-conversion -Wshadow -Wconversion -Wdouble-promotion -Werror (clang++ 19.1; g++ 14.2 takes the same set without the clang-only -Wimplicit-int-float-conversion). Only JUCE's headers are -isystem; third_party/jidai-common is checked like SHOGUN's own code and gives 0 warnings. Before the fix: clang++ 234 unique warning sites (71 -Wdouble-promotion, 65 -Wsign-conversion, 60 -Wmissing-prototypes, 21 -Wimplicit-int-float-conversion, 16 -Wfloat-equal, 1 -Wunused-const-variable kNoRing), g++ 169 (66 -Wsign-conversion, 53 -Wfloat-equal, 29 -Wdouble-promotion, 21 -Wconversion). After: 0 and 0. Every fix spells out a conversion the compiler already made (static_cast), routes an intentional exact compare through dsp::exactEq / tu::same (std::equal_to, same result as ==), declares the web exports in web/wasm/shogun_web.h, or drops the unused kNoRing. Bit-identity: a scratch render of a busy patch (every voice, wave shaper with VC, buses, master, swing, ratchets, flams, micro-timing, probability, locks, three mod rows) at 44.1 and 48 kHz and 1×/2×/4× (4.2 s each, 37.1 MB of doubles) has the same SHA-256 before and after, with g++ and with clang++; the full test log and the web parity output are byte-identical.

Argument-evaluation-order audit (jidai-common test bug fixed upstream in 7eaa5ec). No SHOGUN site needs rewriting: the hits are const getters, ternary branches, short-circuit chains or nested calls, which are sequenced. The g++ and clang++ suites print identical output.

jidai-common's own tests at 8a4b5ae, with the upstream CMake flags plus -O2, g++ and clang++ (identical output):

- CommonTests.cpp: 190 checks, 0 failed (-std=c++17 -O2 -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wno-sign-conversion -Werror).
- DspTests.cpp: 55 checks, 0 failed, at -O2 and -O0 -g (-std=c++17 -O2 -Wall -Wextra -Wpedantic -Wshadow -Werror). 2× ADAA alias -121.8 dB; skip per block: worst sample-to-sample step 0.00398 (per-sample skip 0.00585, sine slope 0.00393); group delay 0.5/1.0/1.5 samples for 1/2/3 running stages (measured 0.5000/1.0000/1.5000).
- HeaderHygiene.cpp: passes (strict set above).

Not run on this box

- forge/tests/test_switch_law.py (make law) needs graphforge (GRAPHFORGE_SRC), which is not installed here; the same switch law is checked natively by testExtBypassIgnoresPattern and testIntIgnoresTrigJacks.
- The web page runs in headless Chrome for web/test_page.mjs only; its audio path (AudioWorklet) is not exercised there.

Repository setup (2026-10-06)

Tree

Public repository: <https://github.com/martialsystems/shogun>

Working name: SHOGUN. A separate product name has not been set. The engine, the clock-switch pin, and the placeholder plate are on branch feat/engine-voices, cut from the design pack in this repository. The pin is in forge/. The plate is in plugin/.

This tree contains:

Path	Role
README.md	Product page: features, install, and build
CHANGELOG.md	Revision list
METHODOLOGY.md	Sources and the quoted choices
SCHEMATICS.md	Equations and 48 kHz rows
BUILD_GUIDE.md	PDF build, and the engine layout
TESTPLAN.md	Named tests
REPO_SETUP.md	This file
engine/dsp.h	Shared STAND-IN helpers
engine/shogun.h	Clock, pattern, knobs, and the process call
engine/shogun.cpp	Voices and the output sum
tests/voices.cpp	Named checks
Makefile	<code>make test</code> builds <code>build/shogun_tests</code> and runs the clock-switch law. <code>make plugin</code> builds the VST3
plugin/CMakeLists.txt	VST3 target. Universal Release build. No standalone target
plugin/Source/	Processor, placeholder plate, knob table, and probe
scripts/build_design_pack_pdf.py	Markdown to PDF
docs/SHOGUN_Design_Pack.pdf	The pack as one PDF
forge/engine_pin.json	GraphForge pin, clock switch only
forge/product_laws.py	Legal INT and legal EXT states passed to <code>require_law</code>
forge/graphs/switch_law.json	The switch law. A Trig cable cannot force EXT
forge/tests/test_switch_law.py	Allows both switch positions and blocks the cable override
LICENSE	All rights reserved, Martial Systems LLC
.gitignore	<code>build/</code> and <code>__pycache__/</code>

The voice engine is framework-free C++ under `engine/`. `process()` does not allocate. There is no panel SVG. JUCE is the wrapper in `plugin/` only.

The source manuals are not in the tree. The German PDF that was read for the quotations stays outside the repository. The pack cites it. It does not redistribute it.

PDF

```
python3 scripts/build_design_pack_pdf.py
```

The script fails if a source file contains an em dash, an en dash, or a heading that defines the work by absence. The PDF is a flowing render of the six markdown files, in the order README, METHODOLOGY, SCHEMATICS, BUILD_GUIDE, TESTPLAN, REPO_SETUP. A title line on each PDF page carries the date 2026-10-06. That date is the revision date. A clock time inside the PDF metadata is only the build time.

Collection link

`martialsystems/jidai-collection` names this repository under the SHOGUN heading on `main`. The rack plugin in that repository compiles BUSHIDO and RONIN. It does not fetch SHOGUN. This engine does not change the rack.

Git

Commits on this repository use author `Martial Systems LLC` and `25778085+martialsystems@users.noreply.github.com`, the same author as the collection repository.

The license grants no right to copy the pack without permission from Martial Systems LLC. Short quotations in METHODOLOGY.md are the disputed parameter sentences, with the source named beside each one.

Clock switch

INT ignores Trig jacks. EXT ignores the pattern. The switch chooses. A plugged Trig cable does not force EXT.

That rule is `forge/graphs/switch_law.json`, loaded by `forge/product_laws.py`. `forge/engine_pin.json` pins GraphForge for this law. `forge/tests/test_switch_law.py` asks `require_law` to allow both switch positions with a cable plugged, and to block a cable that forces EXT, an INT state that reads Trig jacks, and an EXT state that fires from the pattern.

The printed 48 kHz rows stay in `tests/voices.cpp`. A later fit may change a row when `SCHEMATICS.md` and that test change together.

INT and EXT are specified in `METHODOLOGY.md` and `SCHEMATICS.md`. The engine checks are `testExtBypassIgnoresPattern` and `testIntIgnoresTrigJacks`.